「コンピュータグラフィックス」
第4回
座標変換：投影変換、座標変換のまとめ

2026/10/19

稲田 徹悟

i039519@shibaura-it.ac.jp

シラバス解説：授業計画

- イントロダクション(9/28)
- 座標変換(10/5,***10/12***,10/19)
- ポリゴンレンダリングエンジン(10/26,***11/9***,11/16,***11/23***)
- 大域照明モデル(11/30,12/7)
- モデリング(12/14,***12/21***)
- アニメーション(1/18)
- 近年の技術動向(1/25)

テーマ課題

テーマ課題

テーマ課題

期末レポート

（太字は演習）

今日の概要

- 座標変換を3回かけて扱います
  - 形状データからの画像生成に必要な処理です
- 3回目は、投影変換と座標変換のまとめ
  - 投影変換は、描画領域を正規化された直方体に変換します
  - 具体的には、投影行列の掛け算とW除算です
  - 座標変換のまとめとして、OpenGLの座標変換を解説します

目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

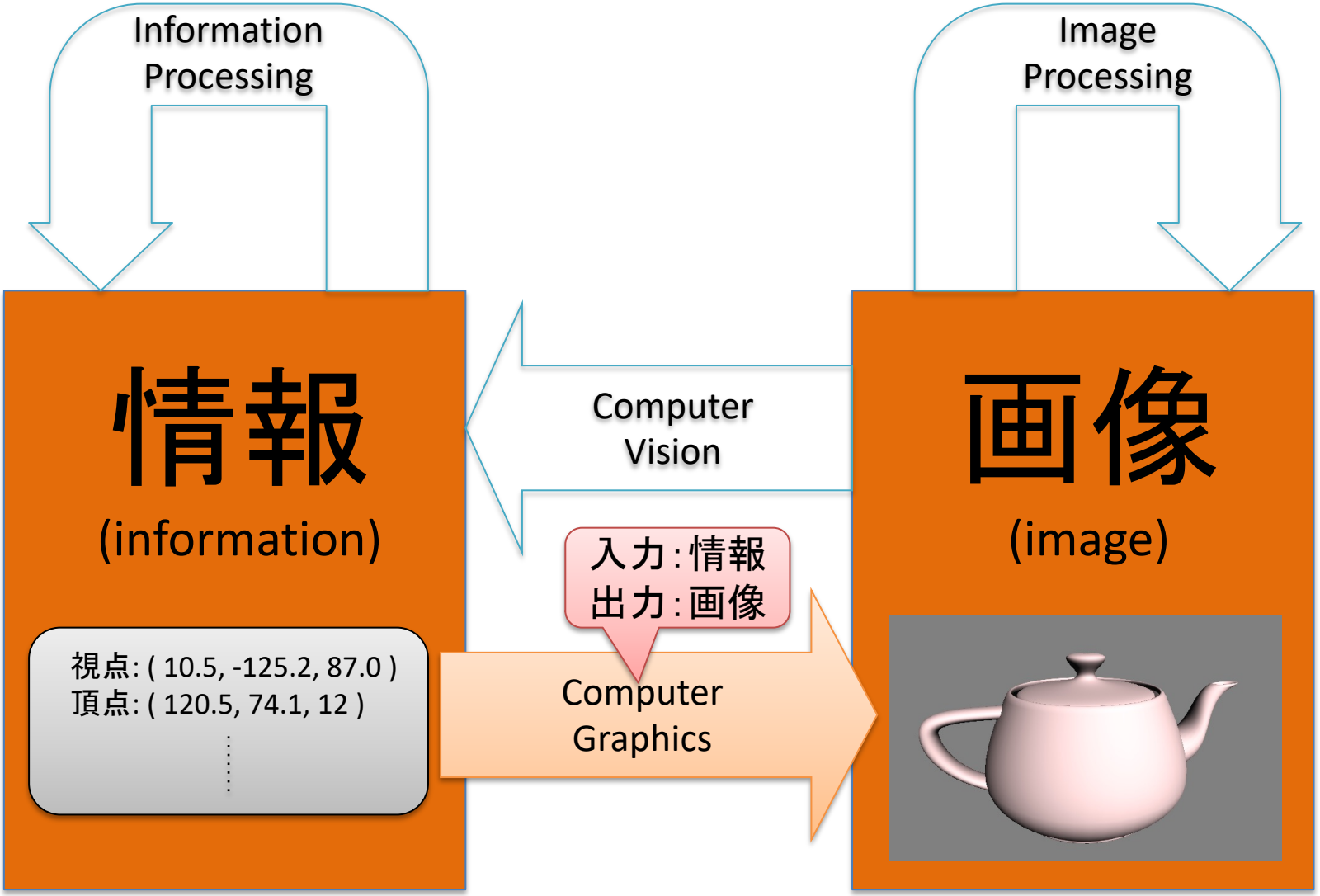
- 単元課題
- ミニッツペーパー

目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

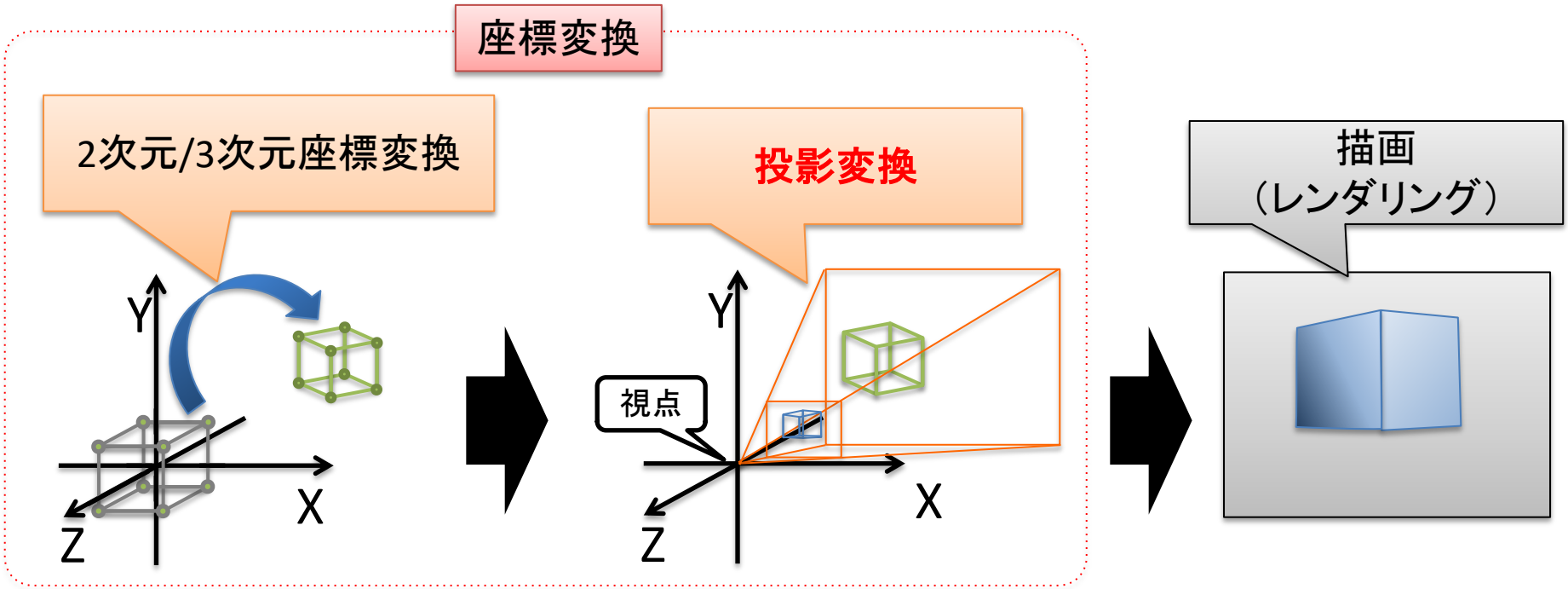
- 単元課題
- ミニッツペーパー

CGは「情報から画像を生成する」



座標変換の目的

- 形状データの描画位置を決める
  - まず、頂点座標を3次元空間の任意の場所に動かす
  - 次に、3次元座標を画像の2次元座標に変換する
  - （その後、画像が描画される）



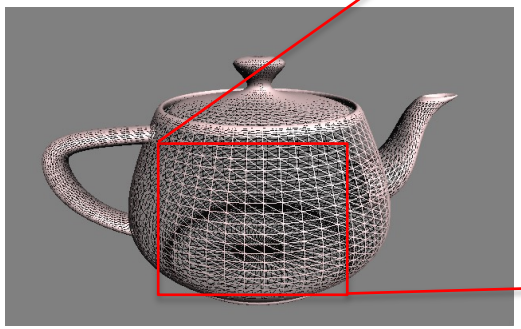
形状データは、座標の集まり

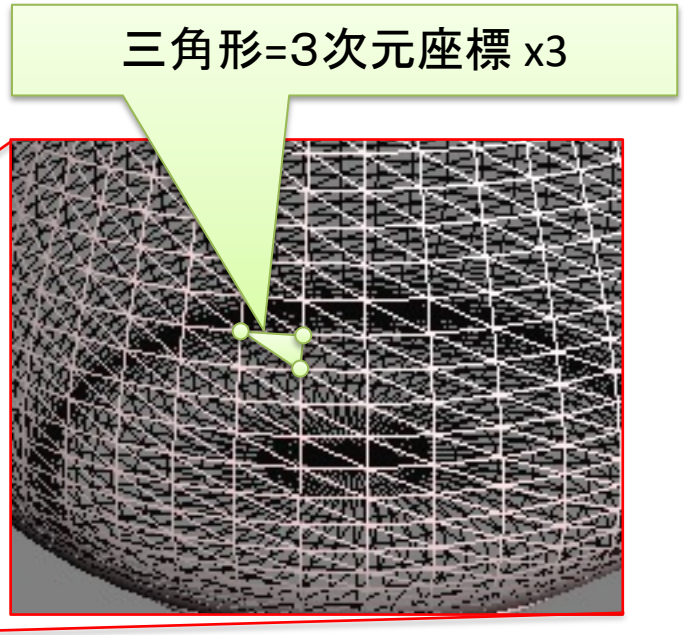
- 形状データを動かす = 頂点座標を動かす
  - 例えば、三角形の場合は3つの頂点座標を動かす
  - 形状データの頂点座標を全て動かせば、形状データが動いたことになる

出力画像



形状データ
(ポリゴンモデル)

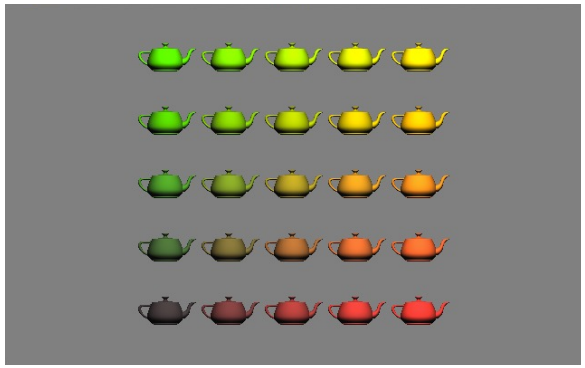


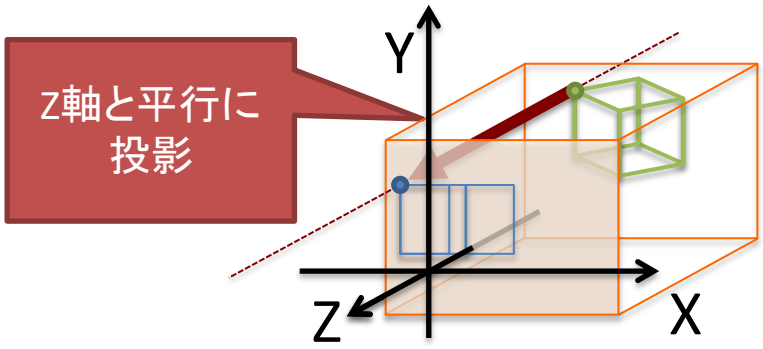


投影変換の概要(1)

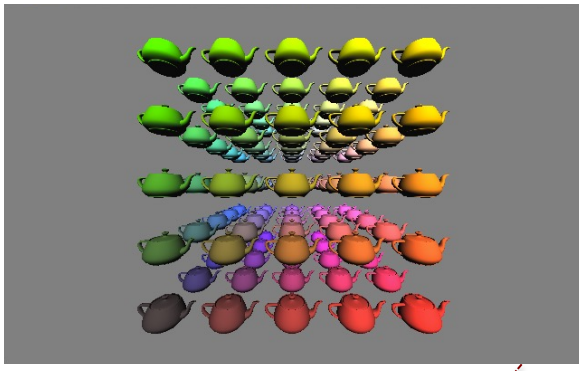
- 3次元座標を2次元座標(+奥行き)に変換する

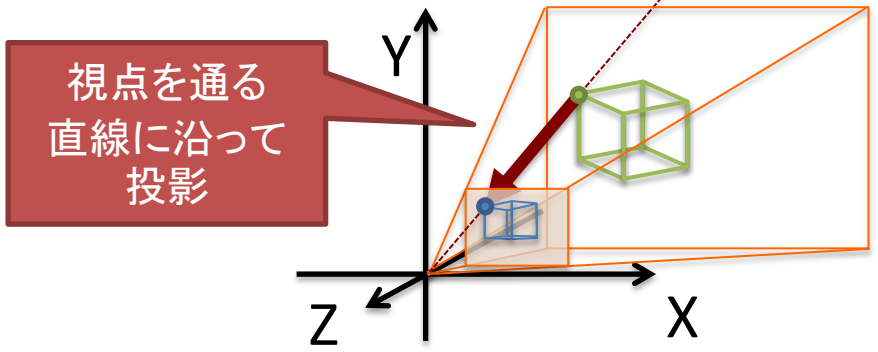
平行投影
(parallel projection)





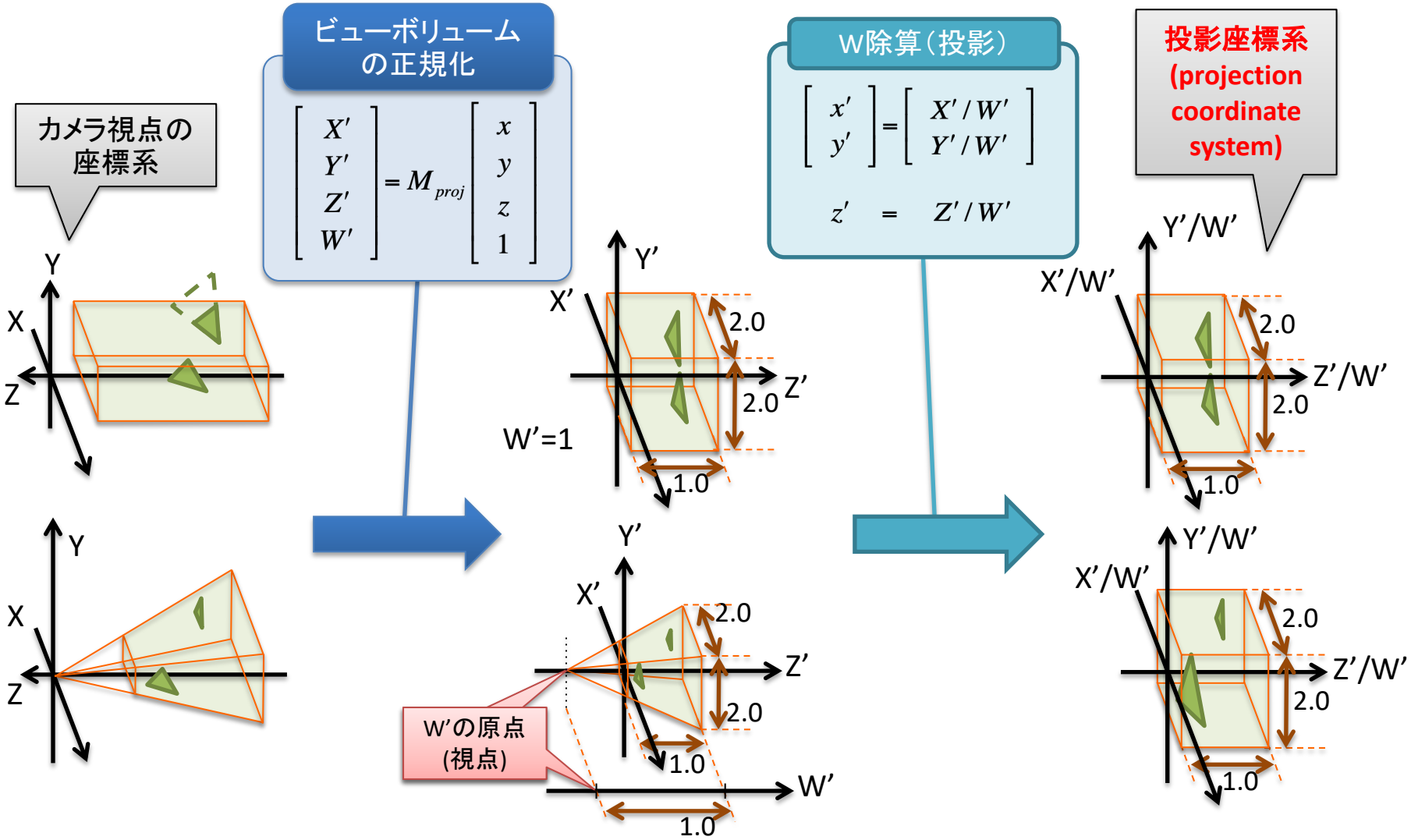
透視投影
(perspective projection)





投影変換の概要(2)

- 投影変換は、ビューボリュームを正規化された直方体に変換する
  - 具体的には、**投影行列(projectin matrix)**の掛け算と、W除算を行う



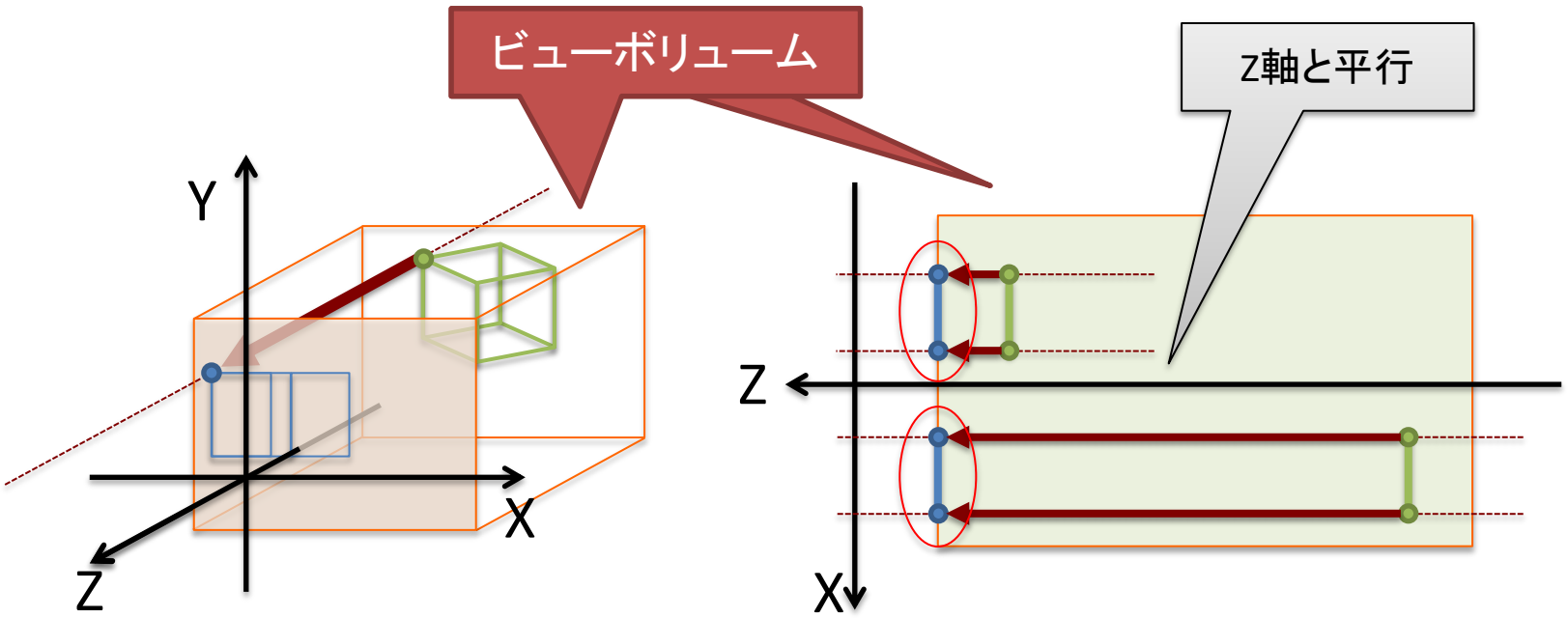
目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

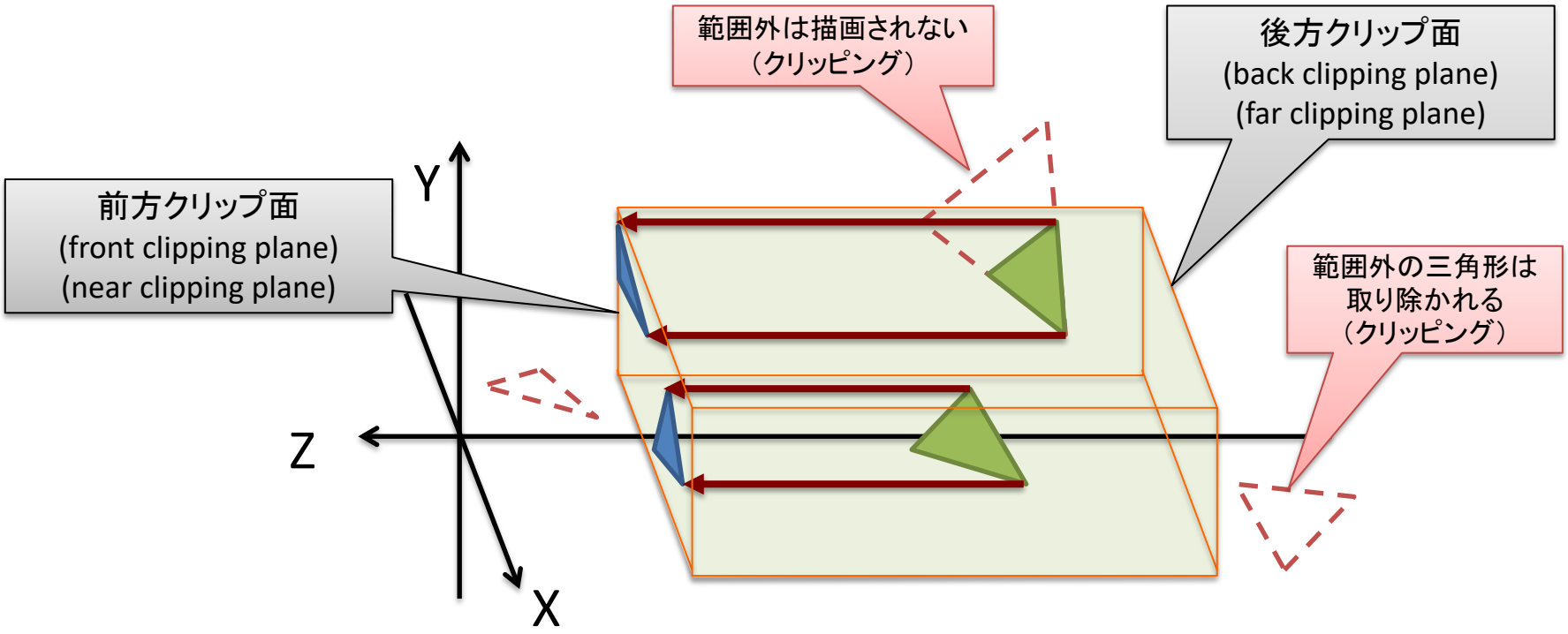
平行投影

- 頂点をZ軸と平行に投影する
  - カメラから近くても遠くても、図形の大きさは変わらない
- 描画範囲が定義される
  - ビューボリューム(View Volume)と呼ぶ



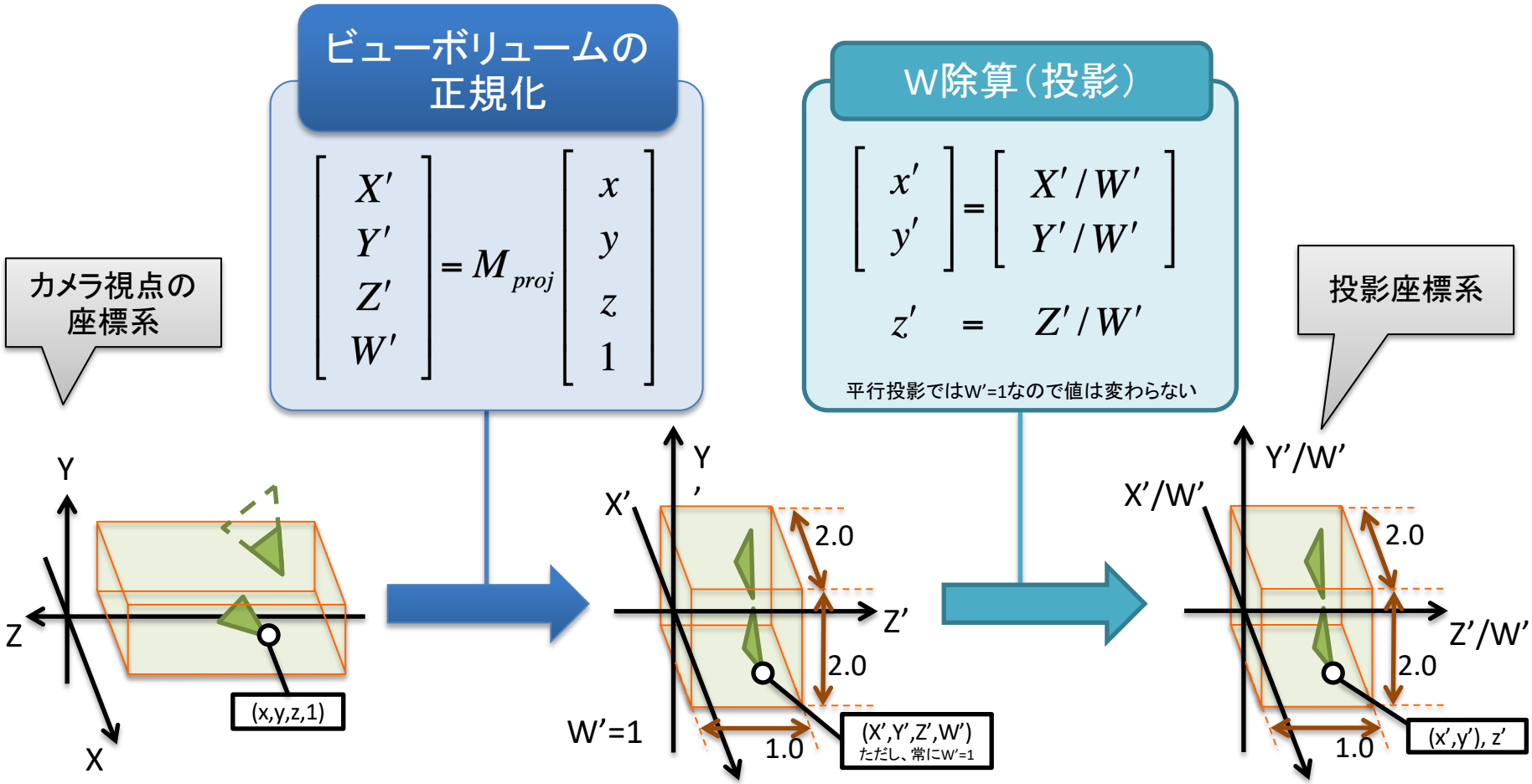
平行投影のビューボリューム

- ビューボリュームは、描画する範囲を定義する
  - ビューボリュームの外は描画されない（クリッピング clipping）
  - 手前と奥の面をクリップ面(clipping plane)と呼ぶ
  - 境界の図形は、ボリューム内の範囲のみ描画される



平行投影の演算

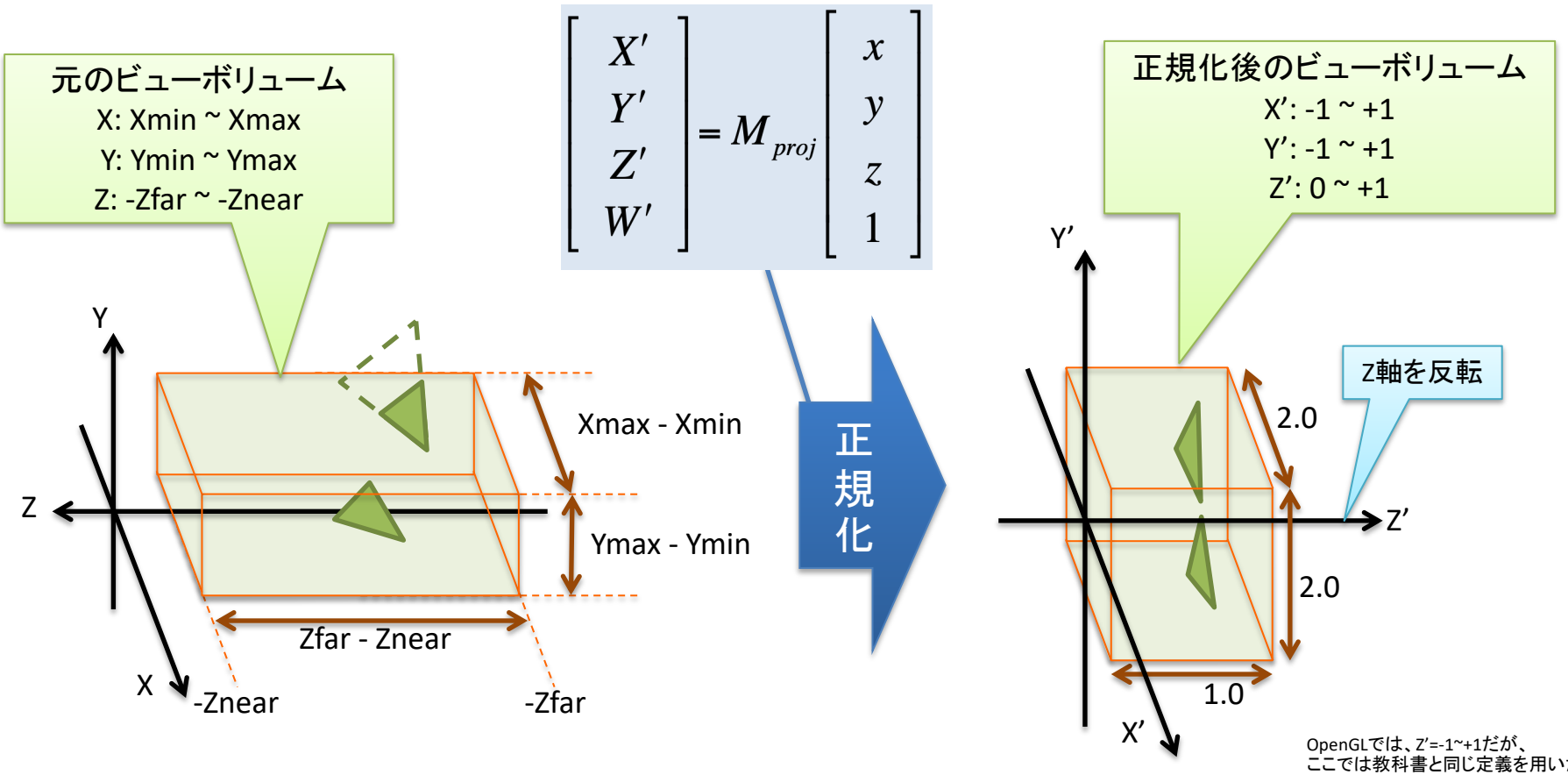
- 平行投影では、ビューボリュームが既に直方体
- 実質の処理はビューボリュームの正規化のみ



ビューボリュームの正規化

- ビューボリュームを正規化された直方体に変換する
  - **投影行列(projection matrix)**を掛ける
  - 奥行き情報を＋で表現するためにZ軸を反転する

$$\begin{bmatrix} X' \\ Y' \\ Z' \\ W' \end{bmatrix} = M_{proj} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



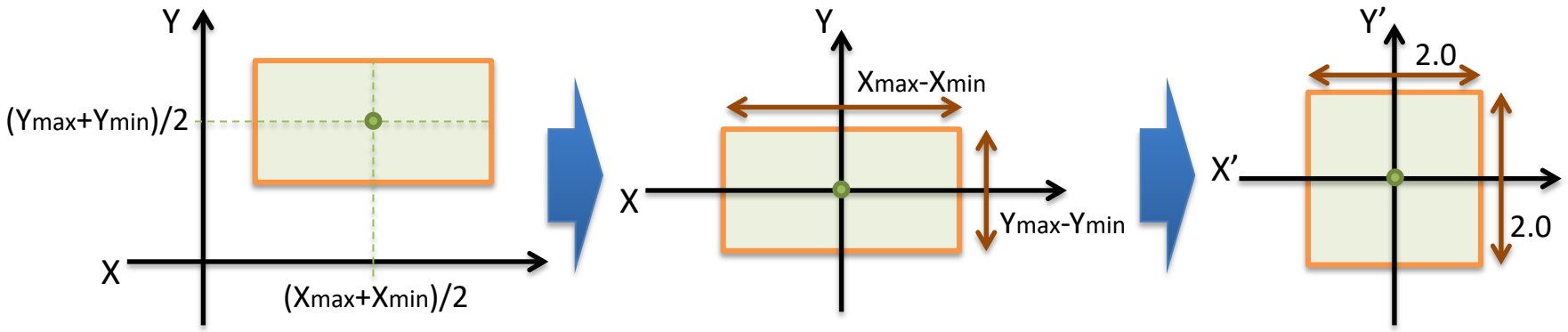
ビューボリュームの正規化(XY座標)

- 任意の長方形を、正方形に正規化する
  - 平行移動と拡大縮小の組み合わせ

2:拡大縮小　　1:平行移動

$$M_{xyproj} = \begin{bmatrix} \frac{2}{x_{max}-x_{min}} & 0 & 0 & 0 \\ 0 & \frac{2}{y_{max}-y_{min}} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -\frac{x_{max}+x_{min}}{2} \\ 0 & 1 & 0 & -\frac{y_{max}+y_{min}}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{2}{x_{max}-x_{min}} & 0 & 0 & -\frac{x_{max}+x_{min}}{x_{max}-x_{min}} \\ 0 & \frac{2}{y_{max}-y_{min}} & 0 & -\frac{y_{max}+y_{min}}{y_{max}-y_{min}} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

元のビューボリューム　　平行移動のみ　　平行移動＋拡大縮小 (=正規化)



ビューボリュームの正規化(Z座標)

- 任意の長方形を、縦横比2:1の長方形に正規化する
  - 平行移動と拡大縮小の組み合わせ

2:拡大縮小&反転

1:平行移動

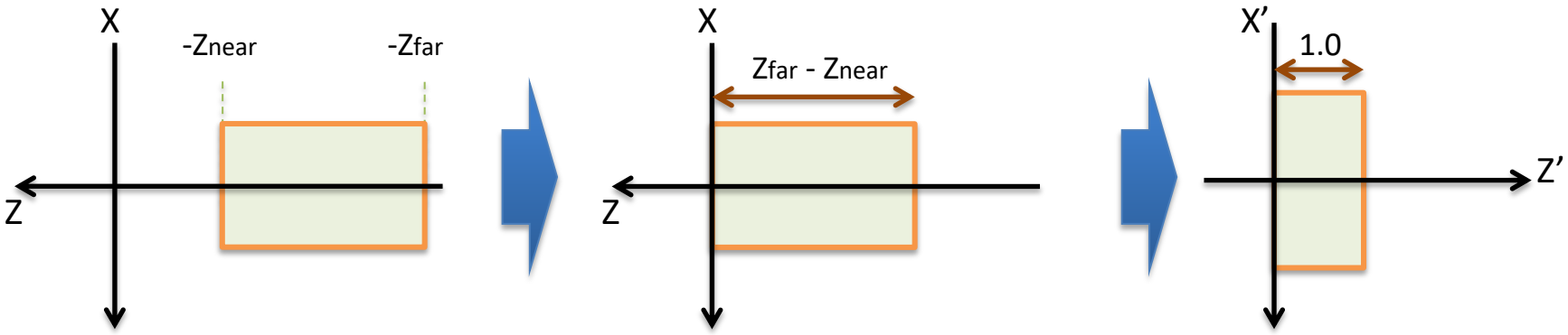
$$M_{zproj} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 * \frac{1}{z_{far} - z_{near}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z_{near} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{1}{z_{far} - z_{near}} & -\frac{z_{near}}{z_{far} - z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

-1を掛けて反転

元のビューボリューム

平行移動のみ

平行移動＋拡大縮小&反転
(=正規化)



ビューボリュームの正規化（投影行列）

- XY座標の投影行列と、Z座標の投影行列とを合成する
  - 軸ごとに独立の処理なので、順序依存はない

$$M_{proj} = M_{zproj} M_{xyproj} = M_{xyproj} M_{zproj}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{1}{z_{far} - z_{near}} & -\frac{z_{near}}{z_{far} - z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{2}{x_{\max} - x_{\min}} & 0 & 0 & -\frac{x_{\max} + x_{\min}}{x_{\max} - x_{\min}} \\ 0 & \frac{2}{y_{\max} - y_{\min}} & 0 & -\frac{y_{\max} + y_{\min}}{y_{\max} - y_{\min}} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

平行投影の投影行列

$$= \begin{bmatrix} \frac{2}{x_{\max} - x_{\min}} & 0 & 0 & -\frac{x_{\max} + x_{\min}}{x_{\max} - x_{\min}} \\ 0 & \frac{2}{y_{\max} - y_{\min}} & 0 & -\frac{y_{\max} + y_{\min}}{y_{\max} - y_{\min}} \\ 0 & 0 & -\frac{1}{z_{far} - z_{near}} & -\frac{z_{near}}{z_{far} - z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

W除算

- 平行投影ではビューボリュームが直方体なので、W除算（投影）は本来必要ない
  - W除算は、後述する透視変換のための処理
- 「投影変換では投影行列を掛けたあとにW除算を行う」という一般化のために、平行投影でもW除算が定義されている
  - 平行投影の投影行列を用いると常に W'=1 になるため、1での除算となり値は変わらない
  - 平行投影と透視投影とで同じ処理にしておくと、ハードウェア化した時に使い回しが効く利点がある

投影行列の乗算

$$\begin{bmatrix} X' \\ Y' \\ Z' \\ W' \end{bmatrix} = \begin{bmatrix} \frac{2}{x_{max}-x_{min}} & 0 & 0 & -\frac{x_{max}+x_{min}}{x_{max}-x_{min}} \\ 0 & \frac{2}{y_{max}-y_{min}} & 0 & -\frac{y_{max}+y_{min}}{y_{max}-y_{min}} \\ 0 & 0 & -\frac{1}{z_{far}-z_{near}} & -\frac{z_{near}}{z_{far}-z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

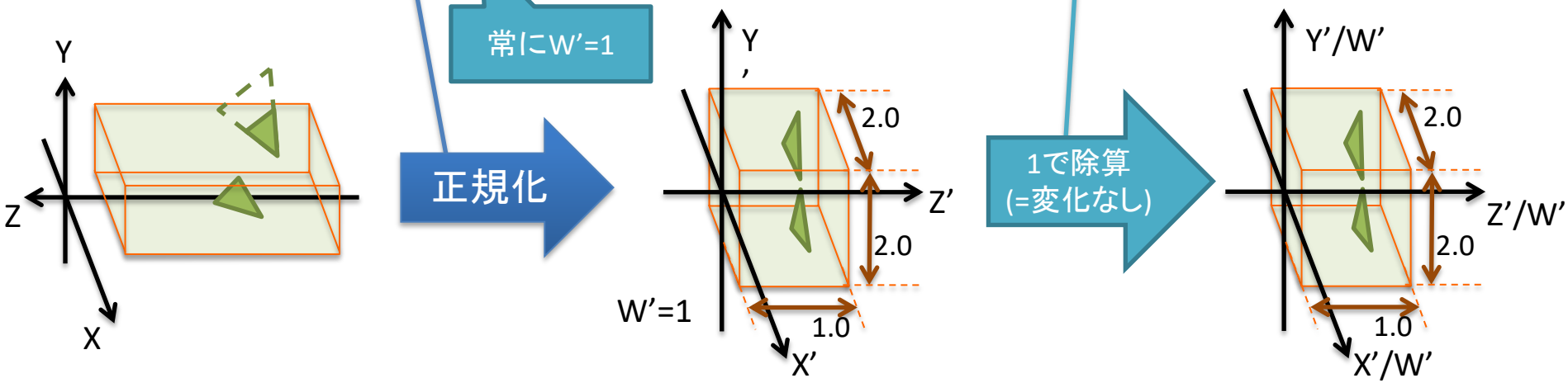
常にW'=1

W除算（投影）

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} X'/W' \\ Y'/W' \end{bmatrix} = \begin{bmatrix} X' \\ Y' \end{bmatrix}$$

$$z' = Z'/W' = Z'$$

平行投影ではW'=1なので値は変わらない



平行投影の演算(まとめ)

- ビューボリュームを、正規化された直方体に変換する
  - 実質的には、ビューボリュームの正規化のみを行う

ビューボリュームの正規化

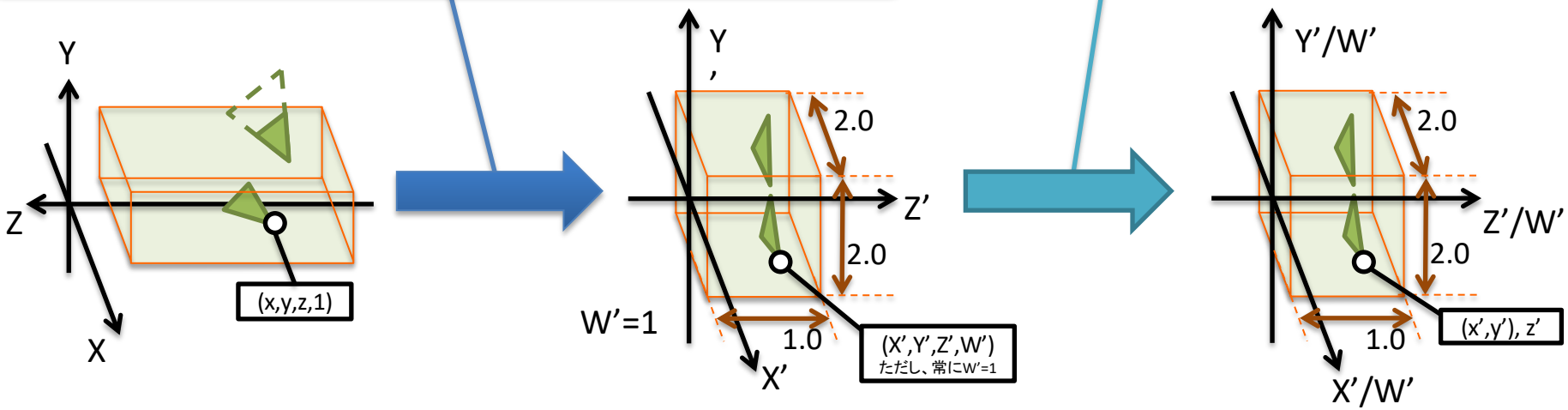
$$\begin{bmatrix} X' \\ Y' \\ Z' \\ W' \end{bmatrix} = \begin{bmatrix} \frac{2}{x_{max}-x_{min}} & 0 & 0 & -\frac{x_{max}+x_{min}}{x_{max}-x_{min}} \\ 0 & \frac{2}{y_{max}-y_{min}} & 0 & -\frac{y_{max}+y_{min}}{y_{max}-y_{min}} \\ 0 & 0 & -\frac{1}{z_{far}-z_{near}} & -\frac{z_{near}}{z_{far}-z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

W除算（投影）

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} X'/W' \\ Y'/W' \end{bmatrix} = \begin{bmatrix} X' \\ Y' \end{bmatrix}$$

$$z' = Z'/W' = Z'$$

平行投影ではW'=1なので値は変わらない



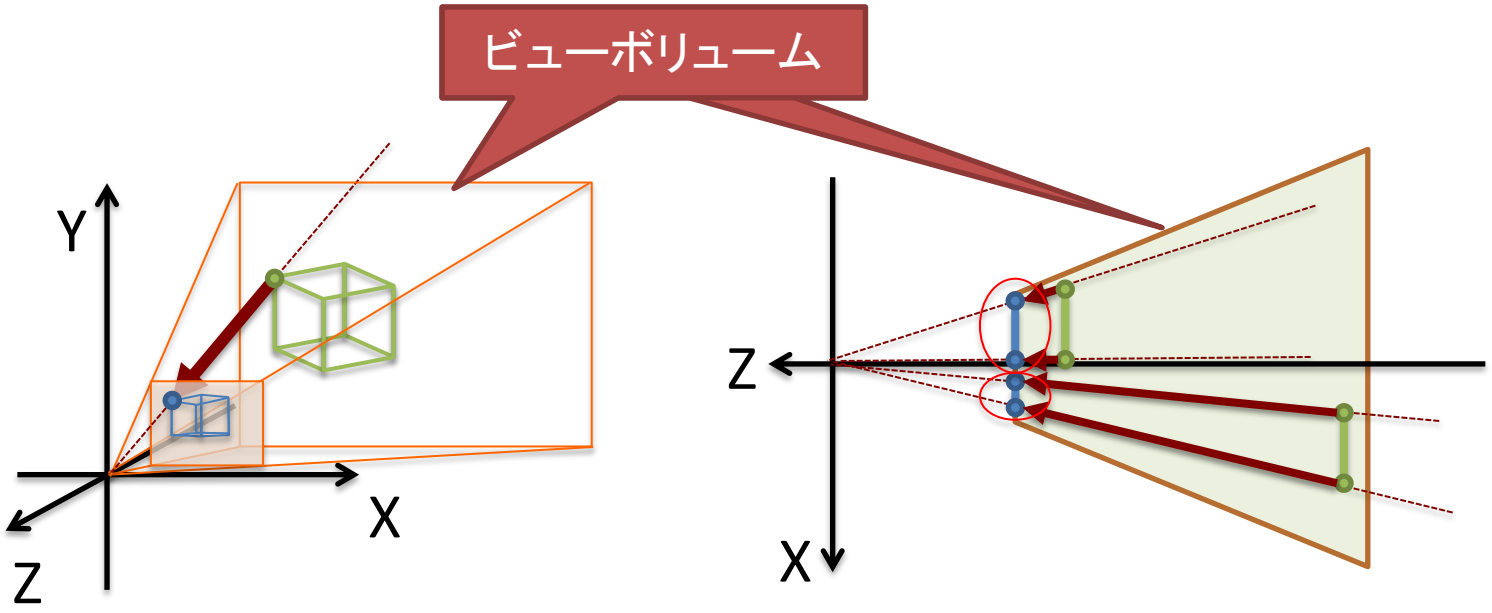
目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

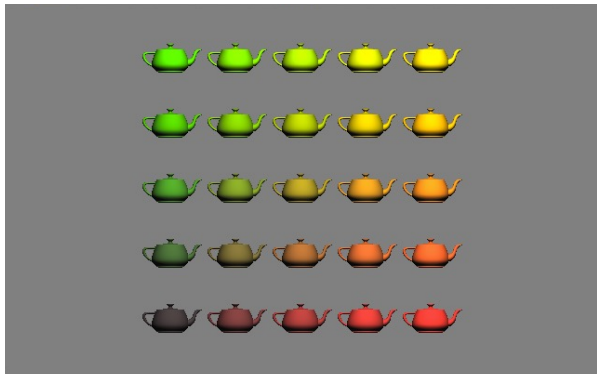
透視投影

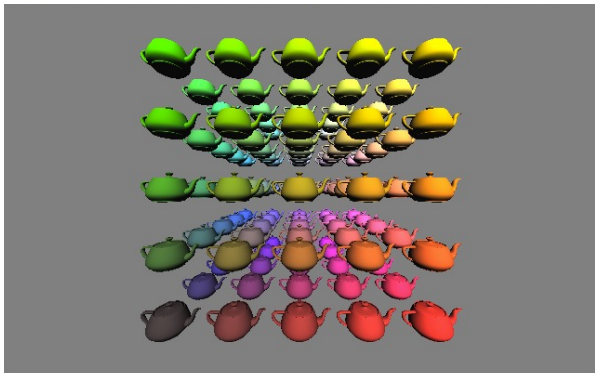
- ビューボリュームを視点と結ぶ直線上に投影する
  - カメラからの距離で、図形の大きさが変化する
  - すなわち、遠近感が表現できる
- 人間の視覚に近いため、広く用いられている

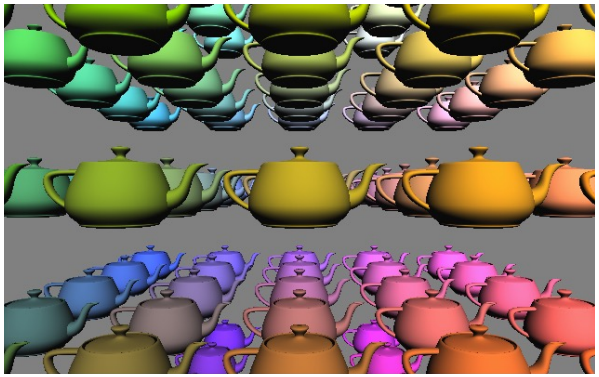


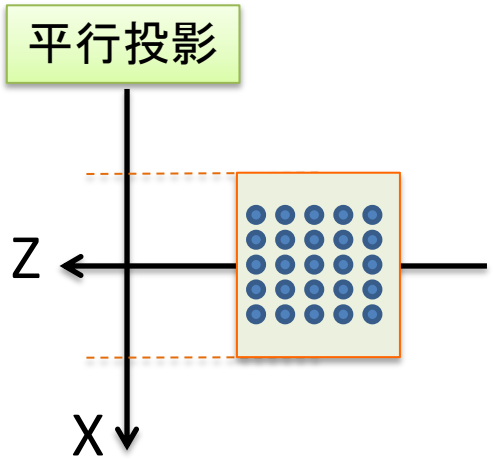
画角による違い

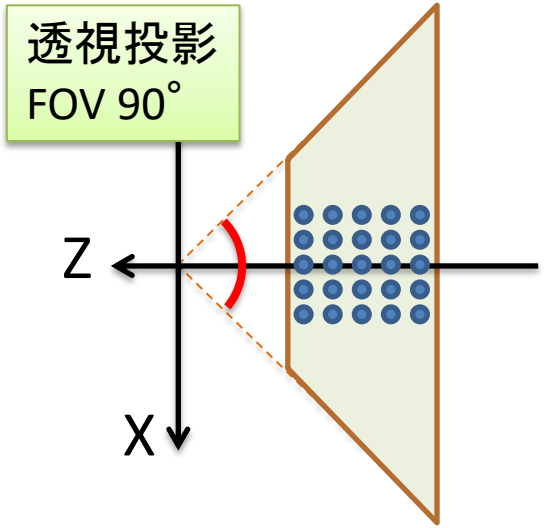
- 同じ位置でも、画角(field of view, FOV)で画像が変わる

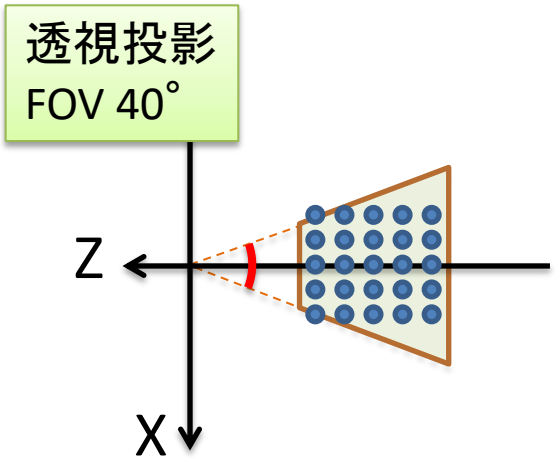






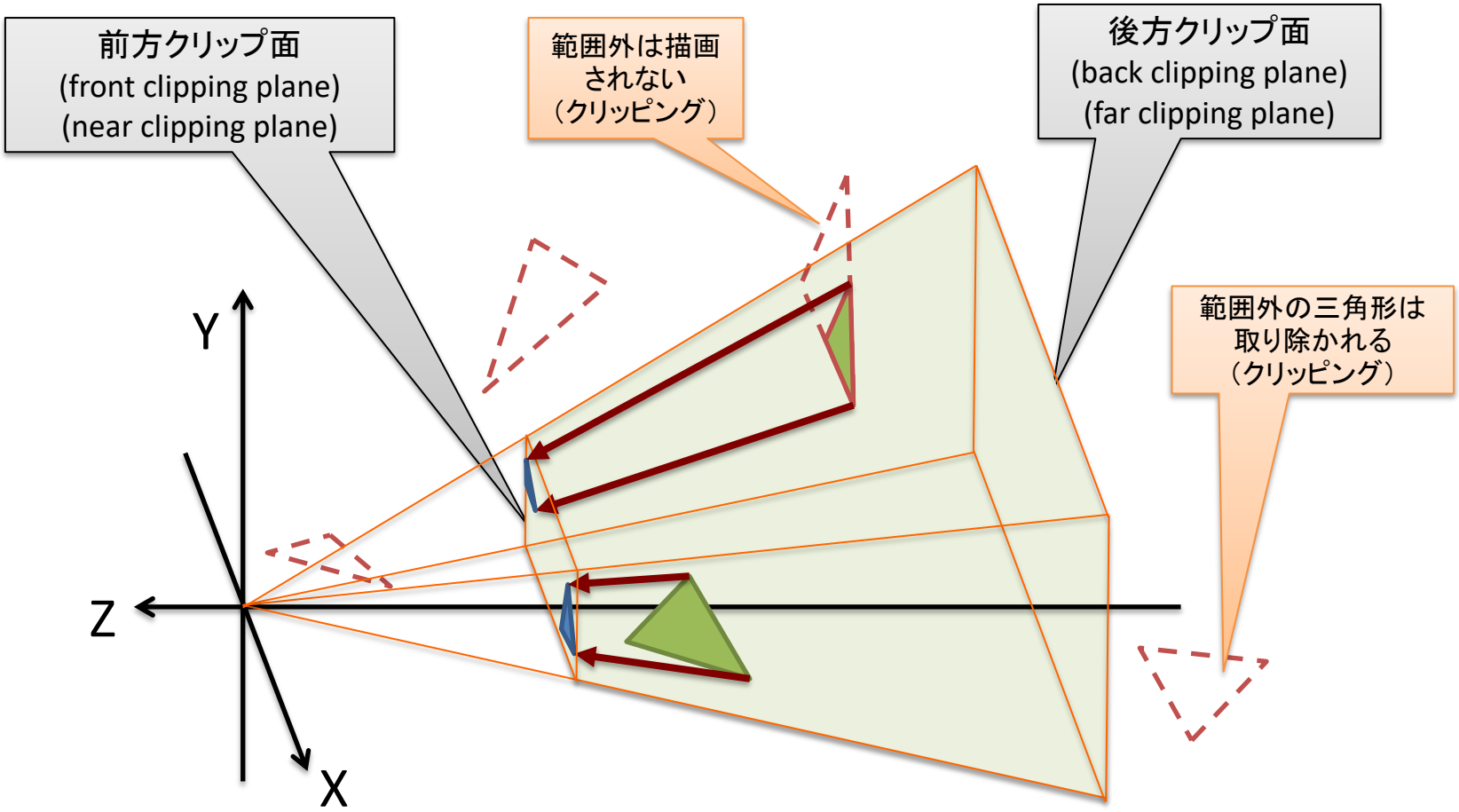






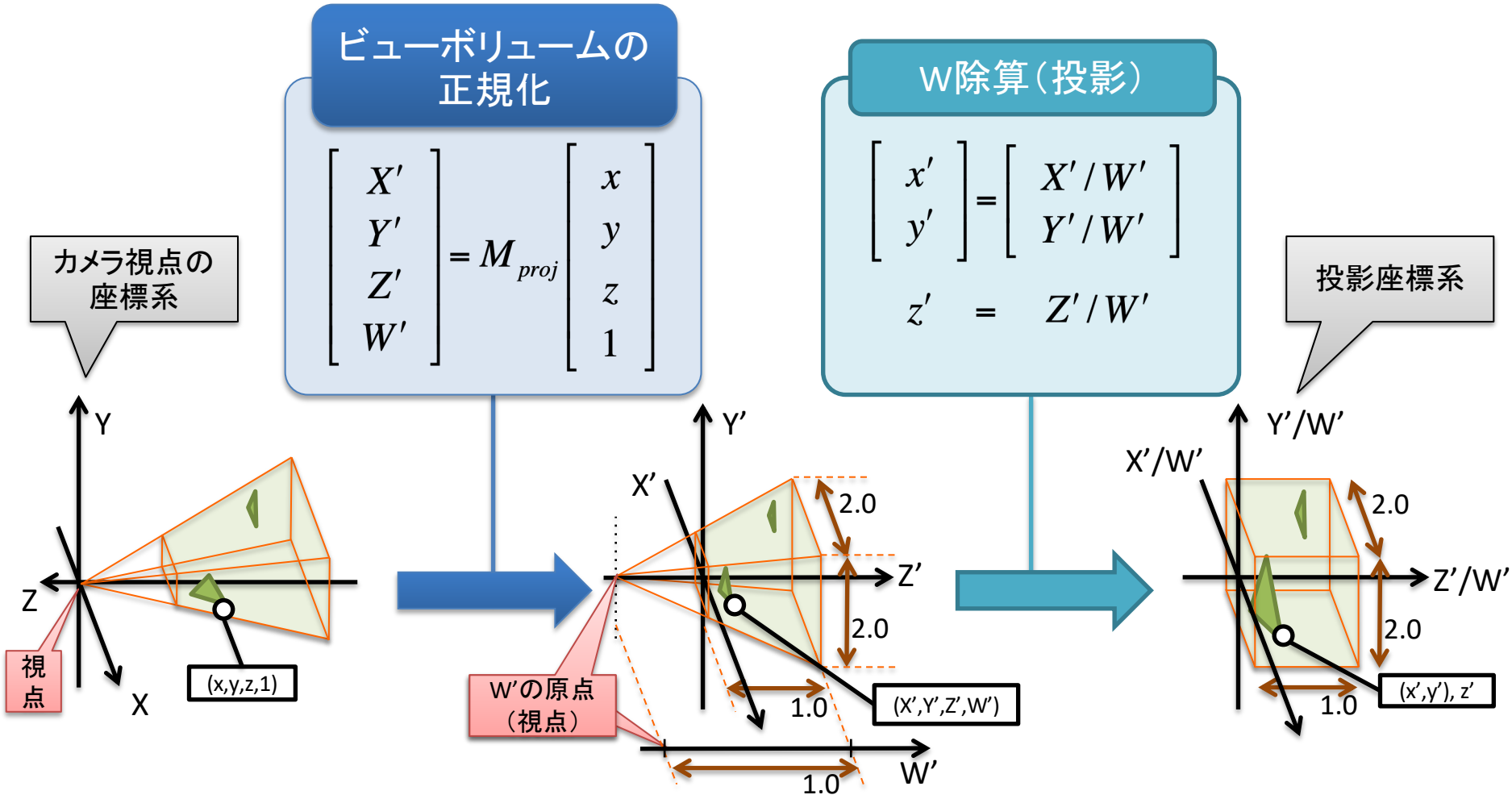
透視投影のビューボリューム

- 視錐台(view frustum)と呼ばれることもある
- 平行投影と同様に、前方後方にクリップ面がある
- 上下左右対称に定義されることが多い
  - 以降、透視投影のビューボリュームは上下左右対称と想定



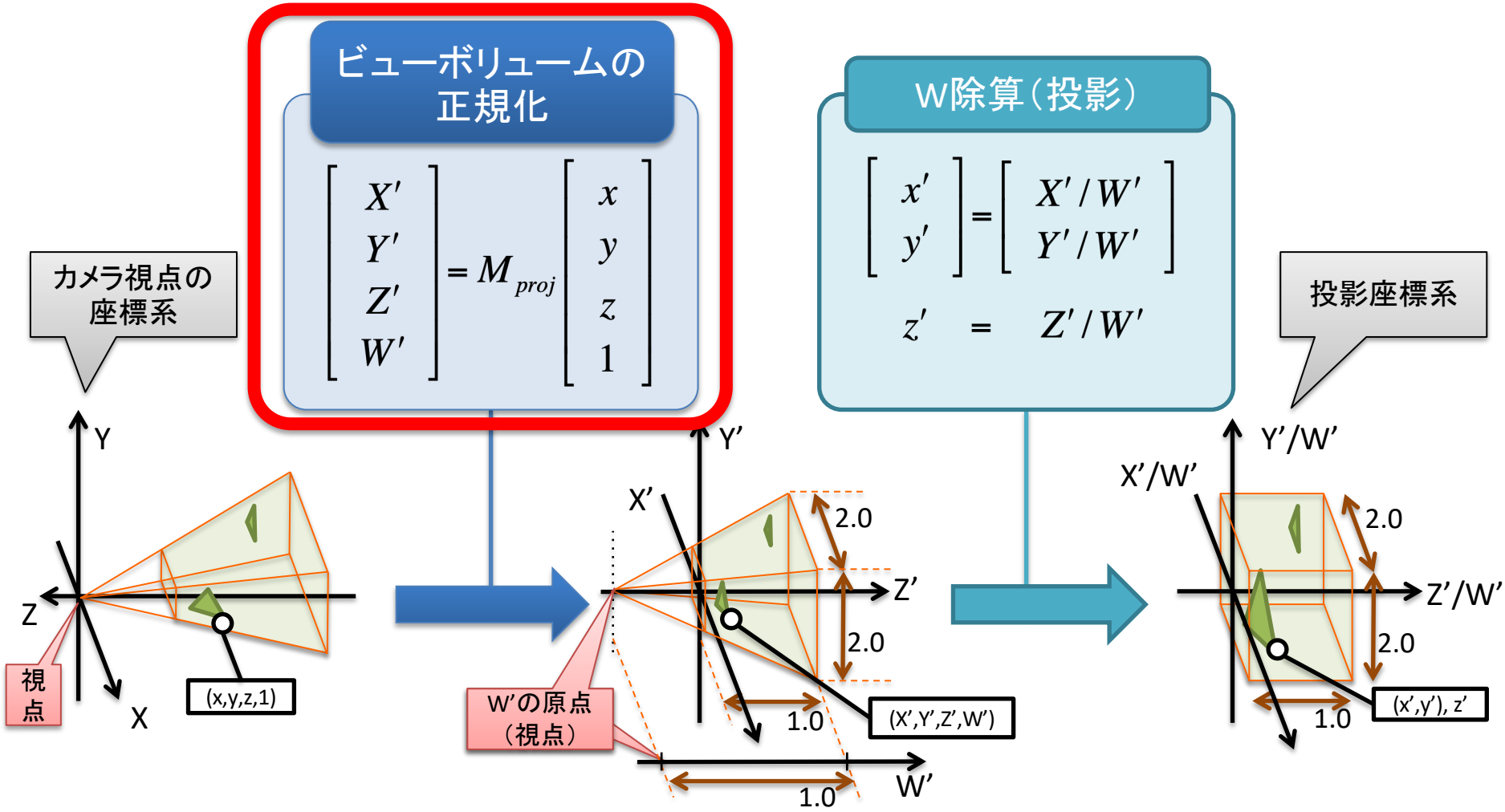
透視投影の演算

- XY平面の正規化は、後方クリップ面を基準にする
- 投影の除算は、視点を基準にした距離(W')で行う
  - 距離(W')による除算で、W'=1.0(後方クリップ面)での座標を計算する



透視投影の演算

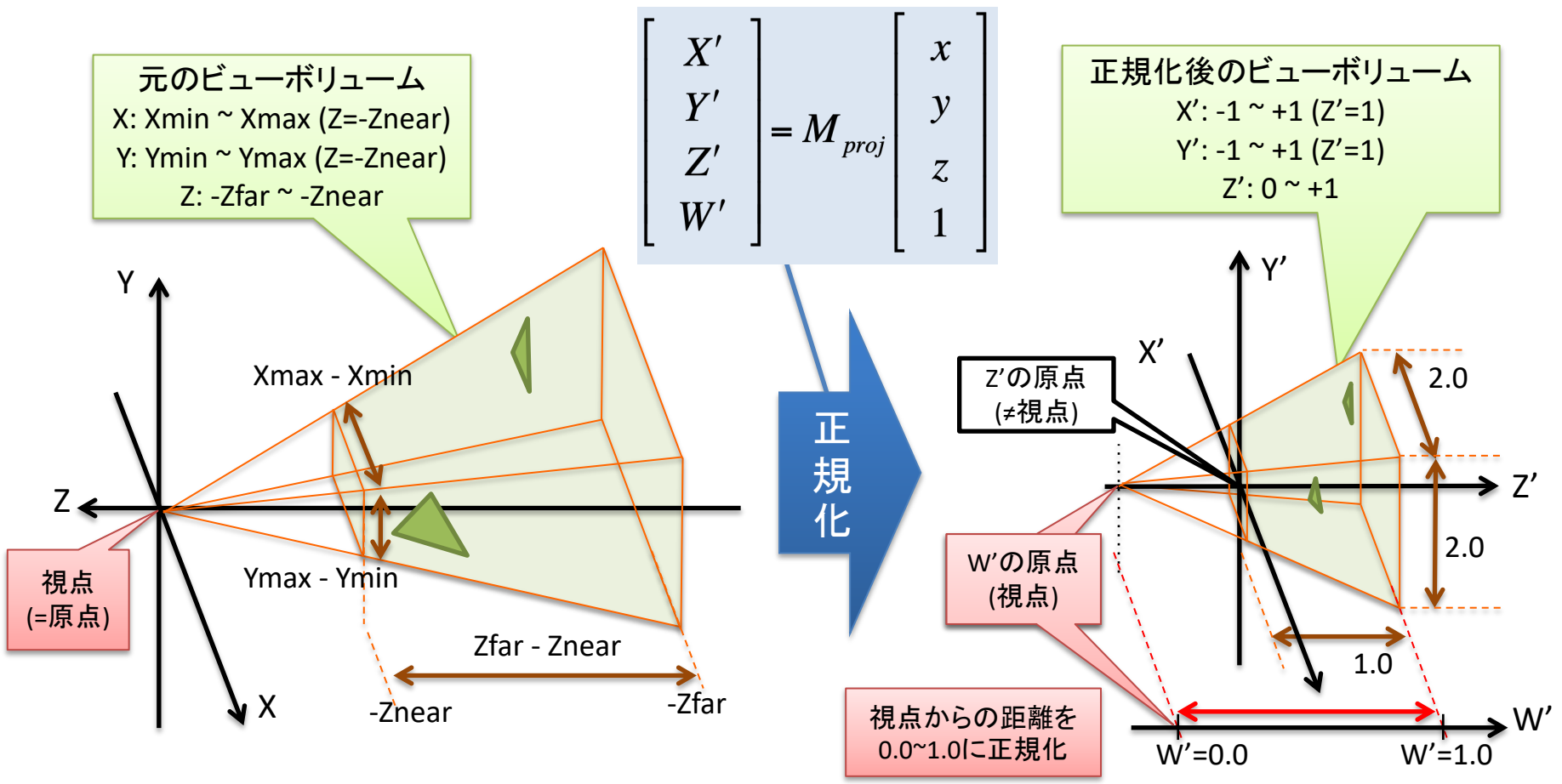
- XY平面の正規化は、後方クリップ面を基準にする
- 投影の除算は、視点を基準にした距離(W')で行う
  - 距離(W')による除算で、W'=1.0(後方クリップ面)での座標を計算する



ビューボリュームの正規化

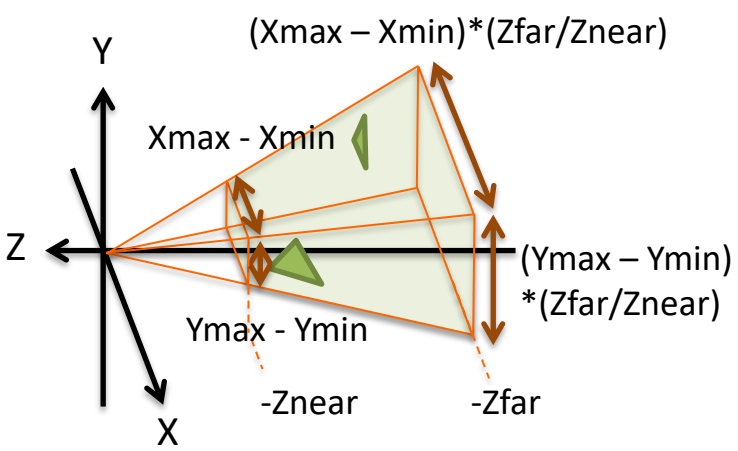
- ビューボリュームを正規化された直方体に内接するように変換する
  - XY平面：後方クリップ面を2.0x2.0に変換する
  - Z座標：-Znear ~ -Zfarを、Z’=0.0~1.0に変換する
  - W座標：相似を保つため、視点からの距離を0.0~1.0に正規化する
    - W軸は、Z軸と向きは同じだが値が異なる

$$\begin{bmatrix} X' \\ Y' \\ Z' \\ W' \end{bmatrix} = M_{proj} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



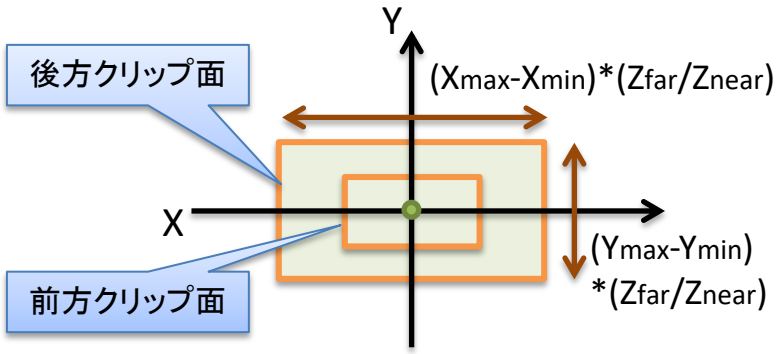
ビューボリュームの正規化(XY座標)

- Z=-Zfarの平面（後方クリップ面）を2.0x2.0に正規化する
  - ビューボリュームが左右対称の場合、拡大縮小のみ

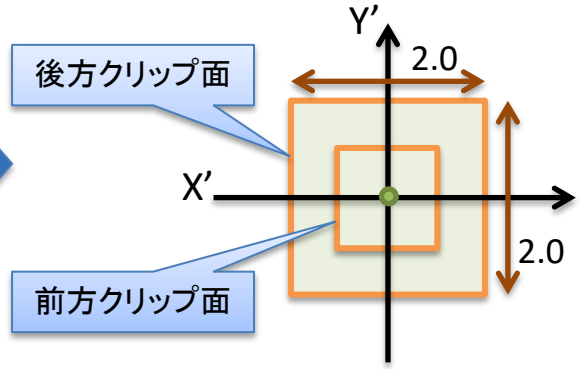


$$M_{xyproj} = \begin{bmatrix} \frac{2}{(x_{max}-x_{min})*\frac{Z_{far}}{Z_{near}}} & 0 & 0 & 0 \\ 0 & \frac{2}{(y_{max}-y_{min})*\frac{Z_{far}}{Z_{near}}} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

元のビューボリューム



拡大縮小(=正規化)



ビューボリュームの正規化(Z座標)

- 任意の台形を、縦横比2:1の長方形に内接するように変換する
  - 平行移動と拡大縮小の組み合わせ

2:拡大縮小&反転

1:平行移動

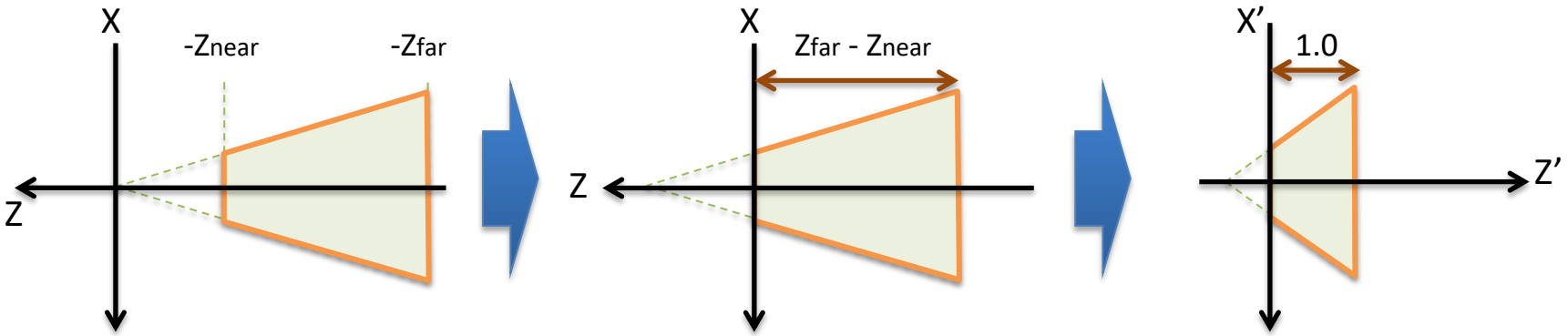
$$M_{zproj} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 * \frac{1}{z_{far} - z_{near}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z_{near} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{1}{z_{far} - z_{near}} & -\frac{z_{near}}{z_{far} - z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

-1を掛けて反転

元のビューボリューム

平行移動のみ

平行移動＋拡大縮小&反転
(=正規化)

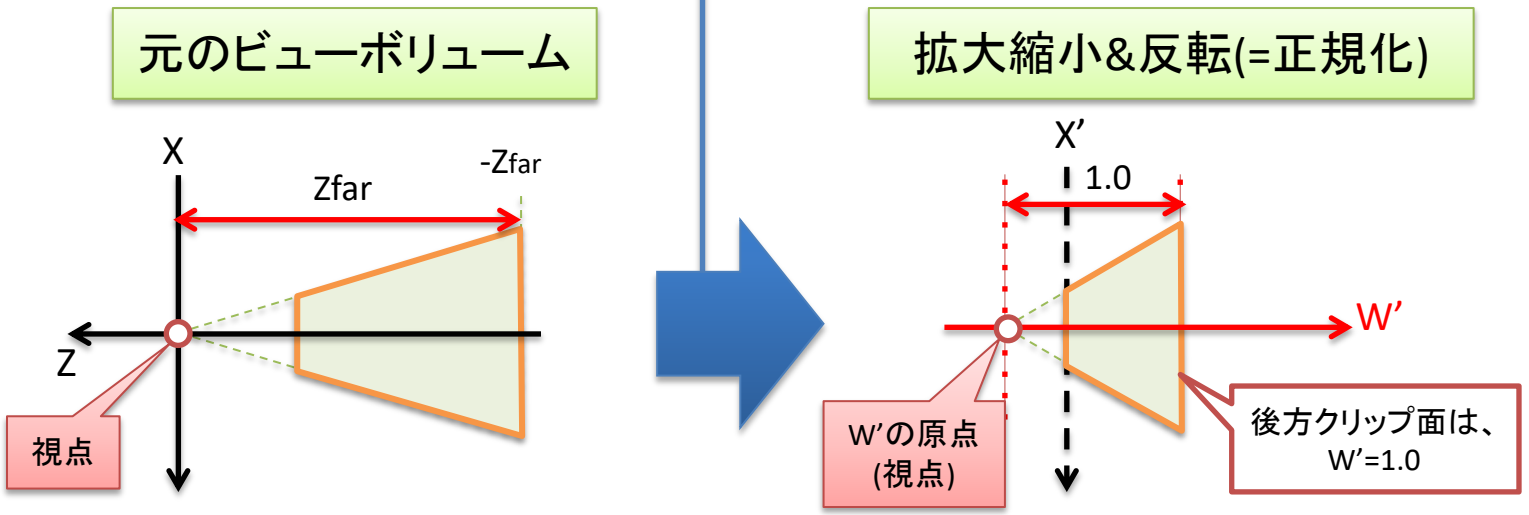


ビューボリュームの正規化(W座標)

- 0~Zfar までの**距離**を、0~+1に正規化する
  - Z座標を反転させて、1/Zfar を掛ける
- 投影のために、視点からの距離を保存するのが目的

$$M_{wproj} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1*\frac{1}{z_{far}} & 0 \end{bmatrix}$$

-1を掛けてZ座標を反転



ビューボリュームの正規化（投影行列）

- XY座標、Z座標、W座標の投影行列を合成する
  - 軸ごとに独立の処理なので、順序依存はない

$$M_{proj} = M_{wproj} M_{zproj} M_{xyproj}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 * \frac{1}{z_{far}} & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{1}{z_{far} - z_{near}} & -\frac{z_{near}}{z_{far} - z_{near}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{2}{(x_{\max} - x_{\min}) * \frac{Z_{far}}{Z_{near}}} & 0 & 0 & 0 \\ 0 & \frac{2}{(y_{\max} - y_{\min}) * \frac{Z_{far}}{Z_{near}}} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

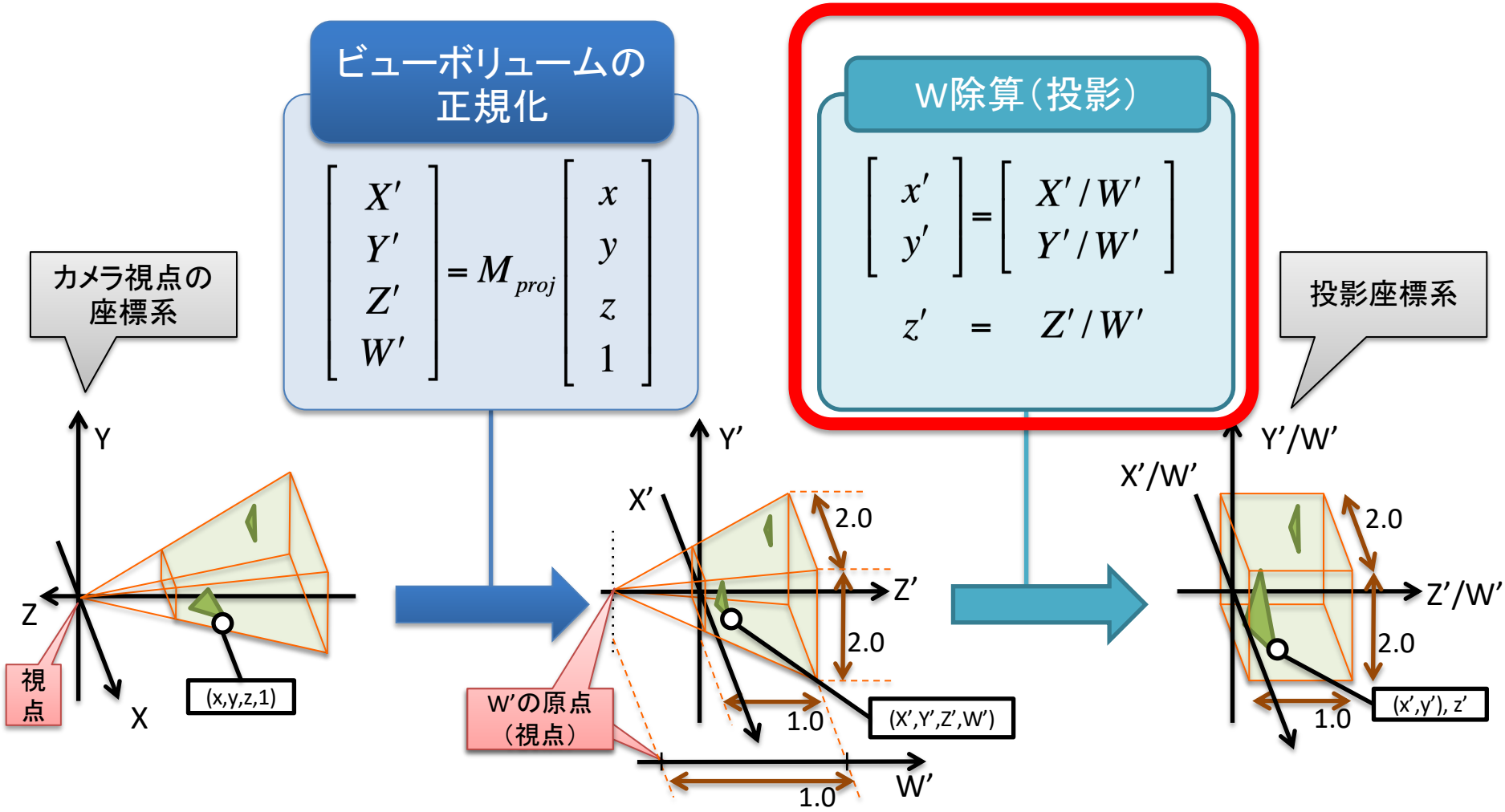
$$= \begin{bmatrix} \frac{2}{(x_{\max} - x_{\min}) * \frac{Z_{far}}{Z_{near}}} & 0 & 0 & 0 \\ 0 & \frac{2}{(y_{\max} - y_{\min}) * \frac{Z_{far}}{Z_{near}}} & 0 & 0 \\ 0 & 0 & -\frac{1}{z_{far} - z_{near}} & -\frac{z_{near}}{z_{far} - z_{near}} \\ 0 & 0 & -1 * \frac{1}{z_{far}} & 0 \end{bmatrix}$$

透視投影の投影行列

教科書 p.45の 式2.32を、d/a = Znear/((xmax-xmin)/2) とおいて展開すると、同じ式になる

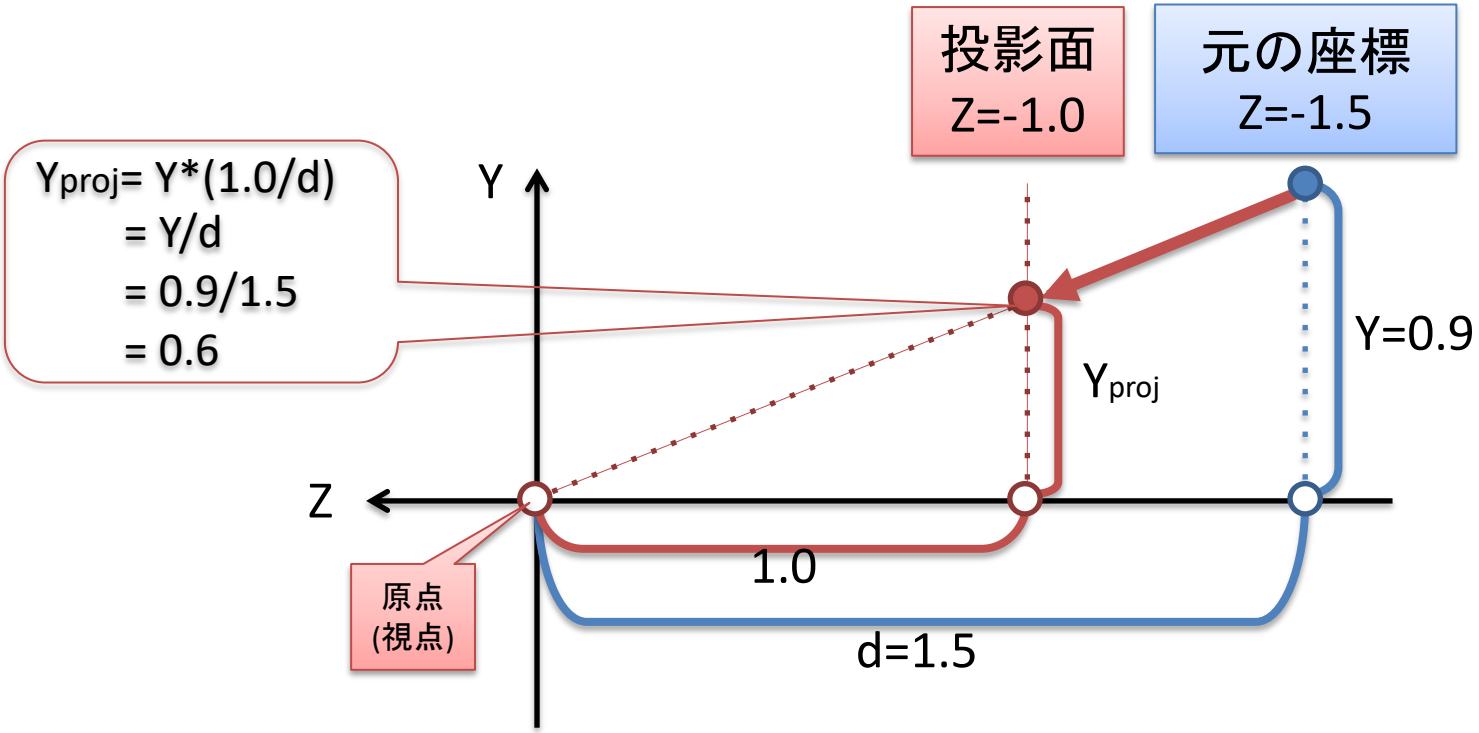
透視投影の演算（再掲）

- XY平面の正規化は、後方クリップ面を基準にする
- 投影の除算は、視点を基準にした距離(W')で行う
  - 距離(W')による除算で、W'=1.0(後方クリップ面)での座標を計算する



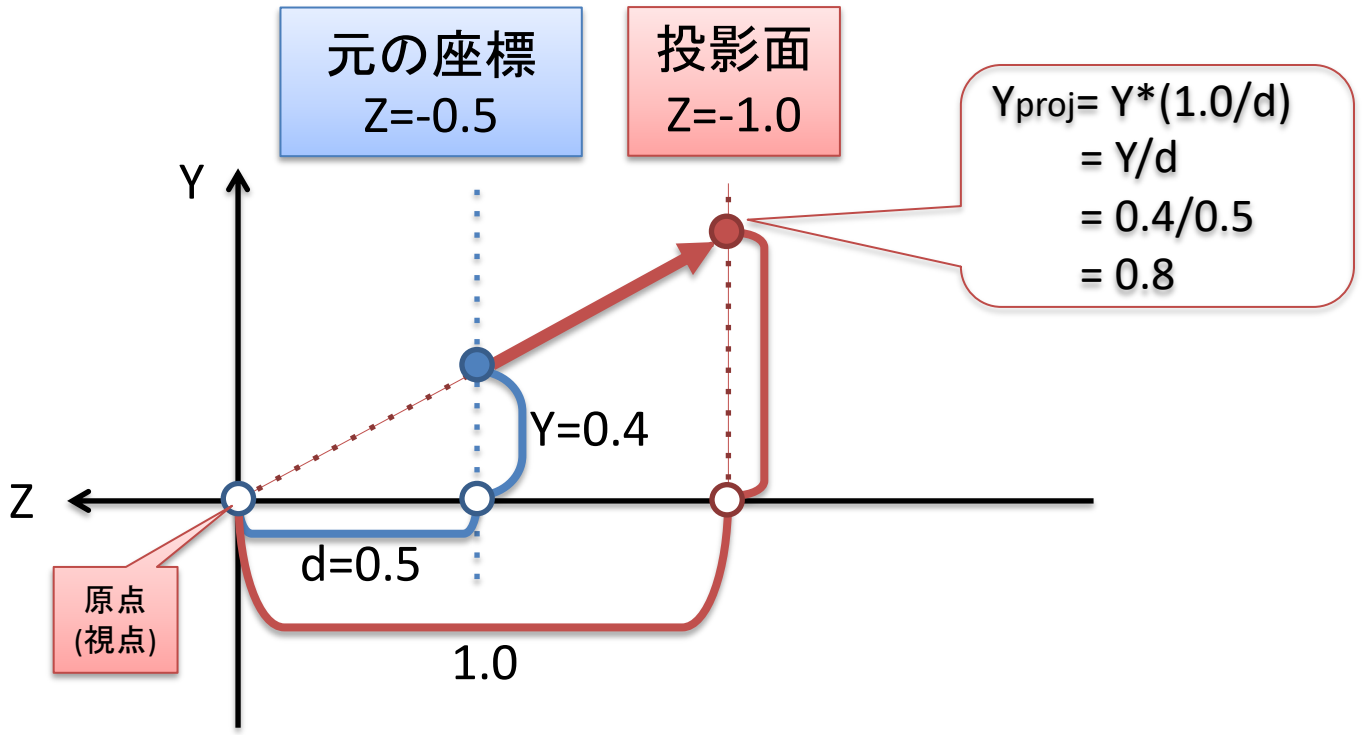
投影演算(1)

- 投影は、三角形の相似を利用する
  - 点、原点、Z軸で構成する三角形を考える
  - 距離=1.0 への投影の場合、距離の除算になる
    - 点のZ軸上の距離をdとすると、三角形の比が 1.0 : d



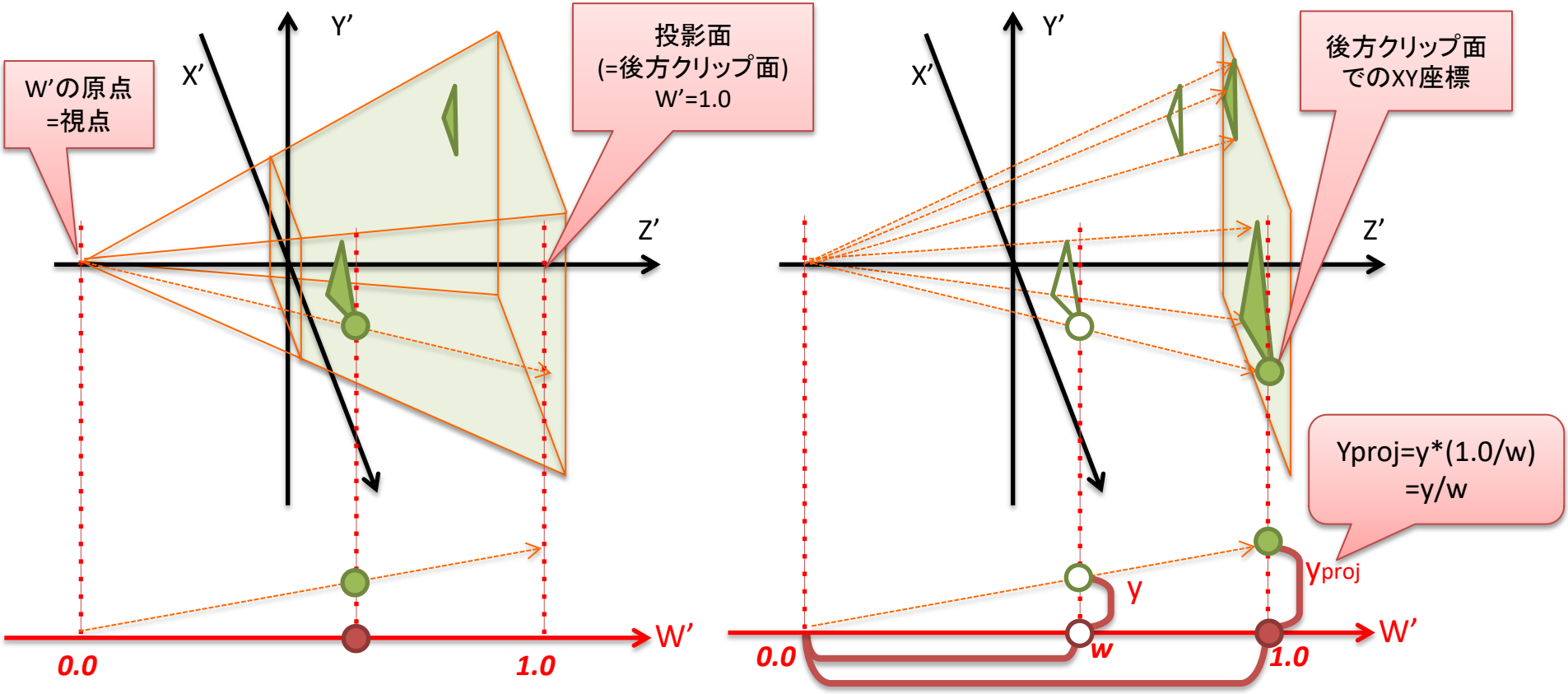
投影演算(2)

- 距離が1.0未満の場合でも、同様に距離の除算で投影できる
  - 手前から奥への投影になる
- よって、「距離=1.0の点の座標」は距離の除算で求められる



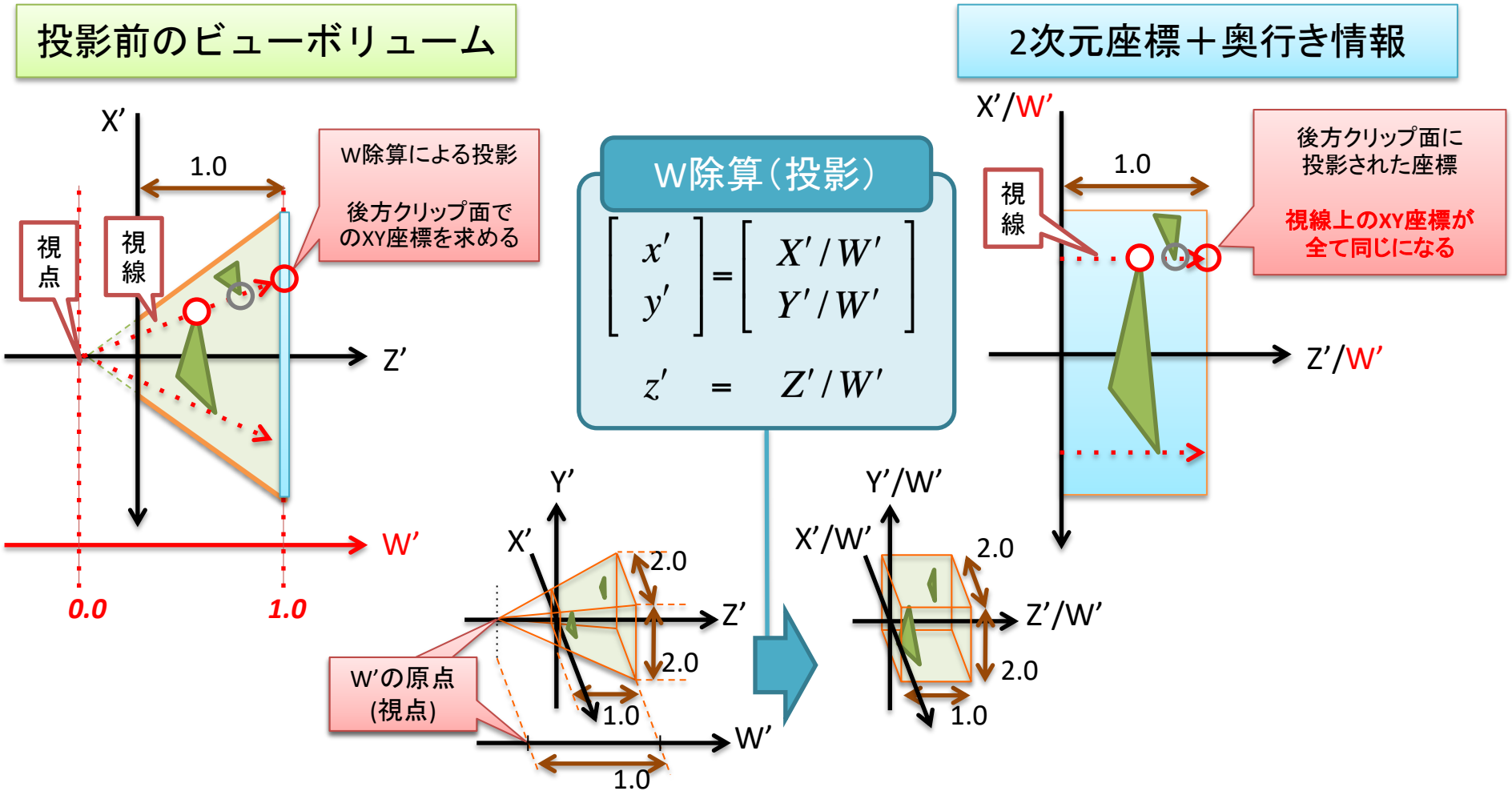
W除算による投影

- W除算は「後方クリップ面に投影する」処理
  - 視点を頂点とした三角形の相似の利用
  - 後方クリップ面は、W'=1.0 の面



W除算によるビューボリュームの変化

- 見た目は3次元だが、実体は「2次元座標＋奥行き情報」
  - 同一視線上の物体は、同じ2次元座標を持つ
  - 近くの物体は拡大される

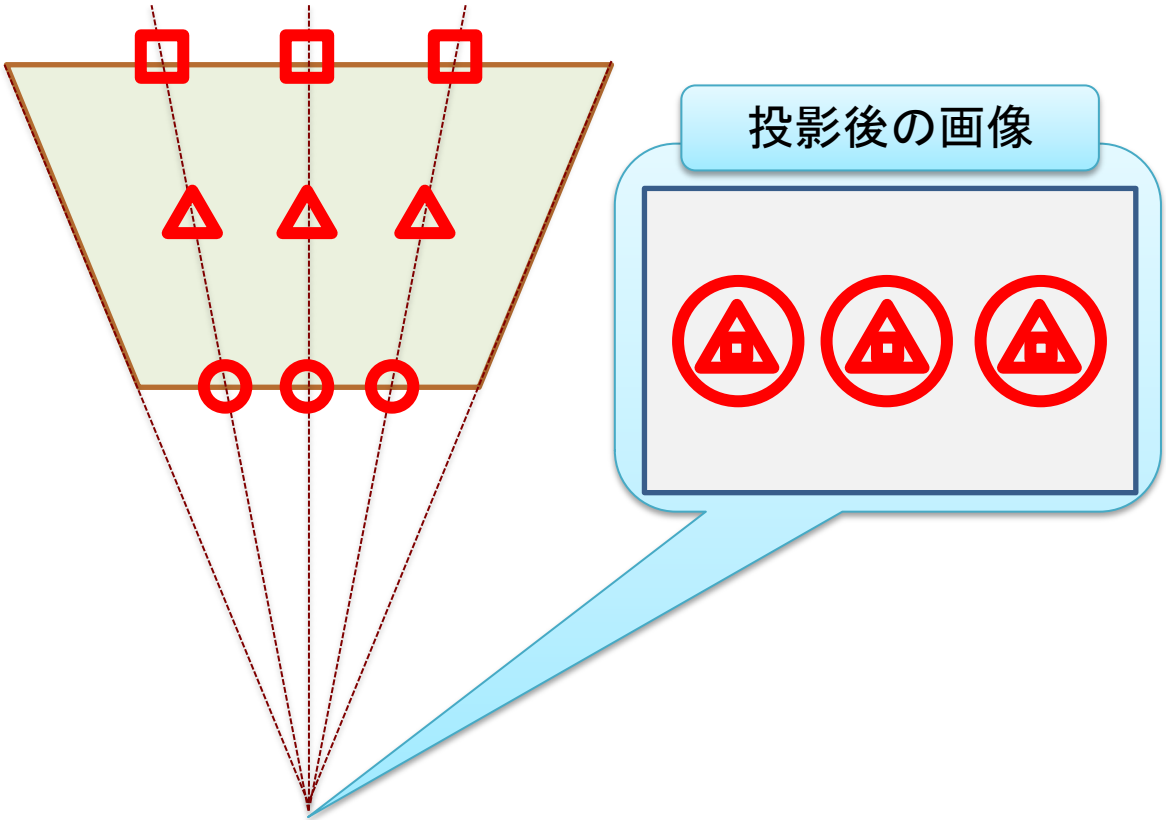


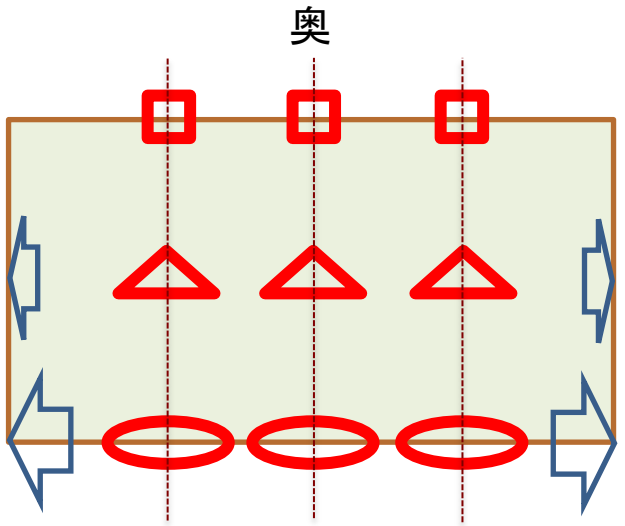
投影の効果

- W除算で座標が奥の面に投影され、手前の図形は拡大される
- この投影で、ボリュームが「2次元座標＋奥行き情報」になる

正規化ビューボリューム

投影後の
「2次元座標＋奥行き情報」





実際は、投影によって奥行きの値も変化するが、ここでの説明では考慮しない

透視投影の演算(まとめ)

- 後方クリップ面を基準に正規化を行う
- 元の視点を基準にした距離(W')で、投影の除算を行う

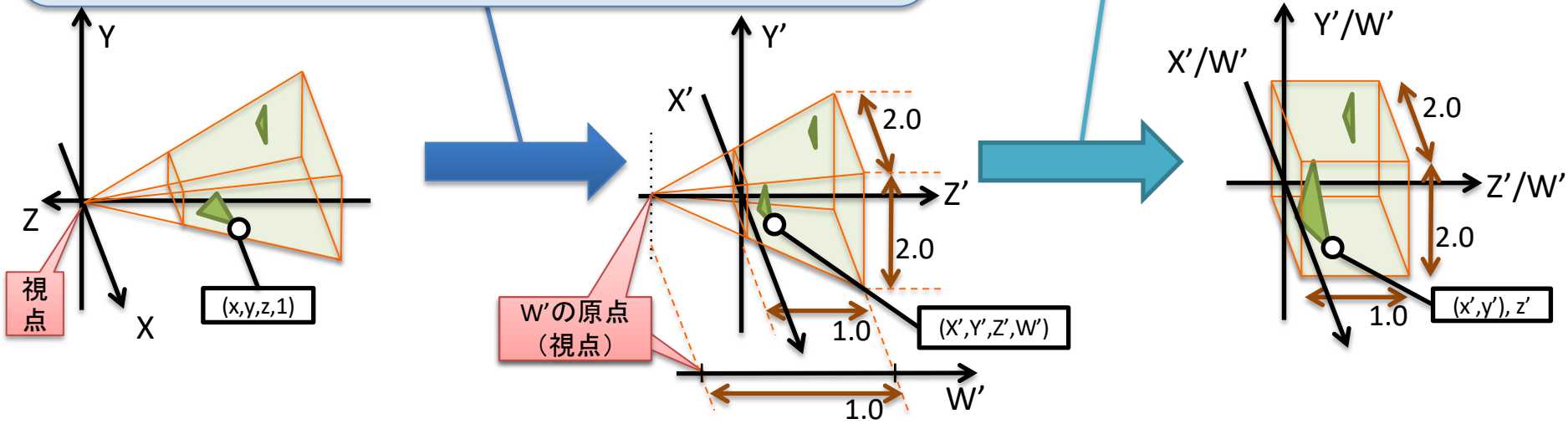
ビューボリュームの正規化

$$\begin{bmatrix} X' \\ Y' \\ Z' \\ W' \end{bmatrix} = \begin{bmatrix} \frac{2}{(x_{max}-x_{min})*\frac{Z_{far}}{Z_{near}}} & 0 & 0 & 0 \\ 0 & \frac{2}{(y_{max}-y_{min})*\frac{Z_{far}}{Z_{near}}} & 0 & 0 \\ 0 & 0 & -\frac{1}{z_{far}-z_{near}} & -\frac{z_{near}}{z_{far}-z_{near}} \\ 0 & 0 & -1*\frac{1}{z_{far}} & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

W除算（投影）

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} X'/W' \\ Y'/W' \end{bmatrix}$$

$$z' = Z'/W'$$



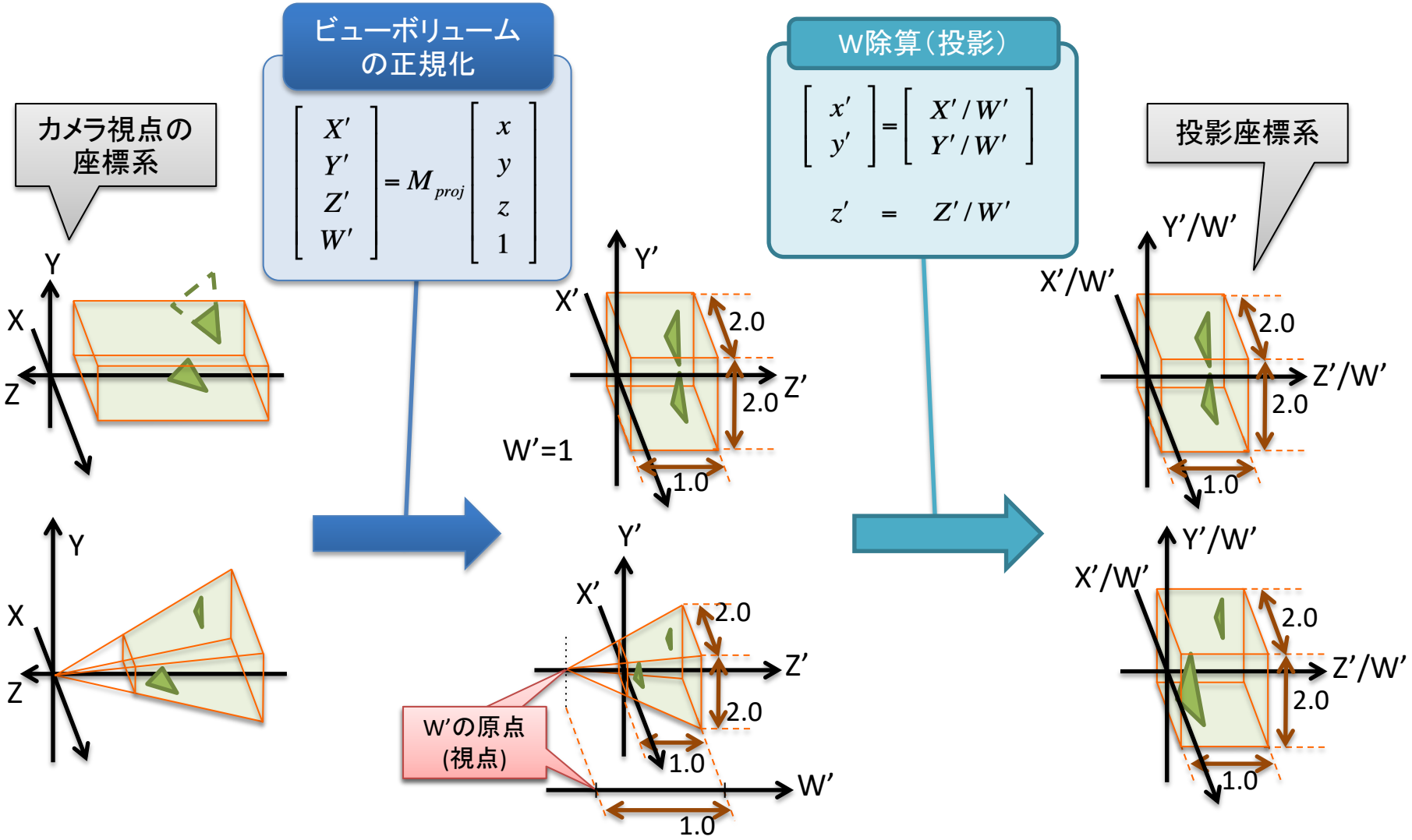
目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

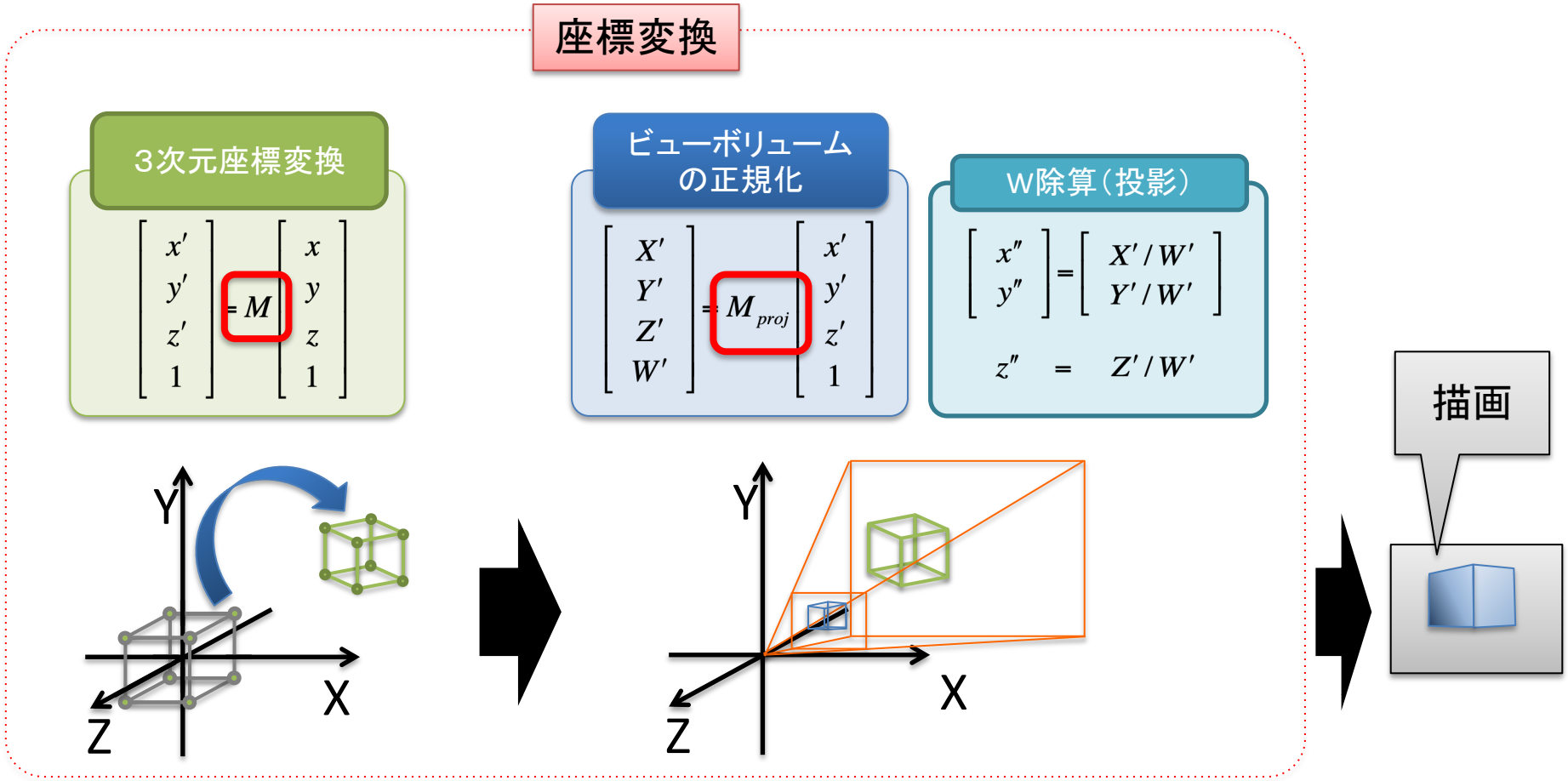
投影変換のまとめ

- 投影変換には、平行投影と透視投影の2種類がある
- 投影変換によって、ビューボリュームを正規化された直方体に変換する
  - 具体的には、投影行列の掛け算と、W除算を行う



座標変換のまとめ

- 3次元座標変換は、行列の積で行う
  - 同次座標によって平行移動、拡大縮小、回転が行列1つに集約できる
- 投影は、行列の積とW除算で行う
  - W除算による投影で、2次元座標が決まる



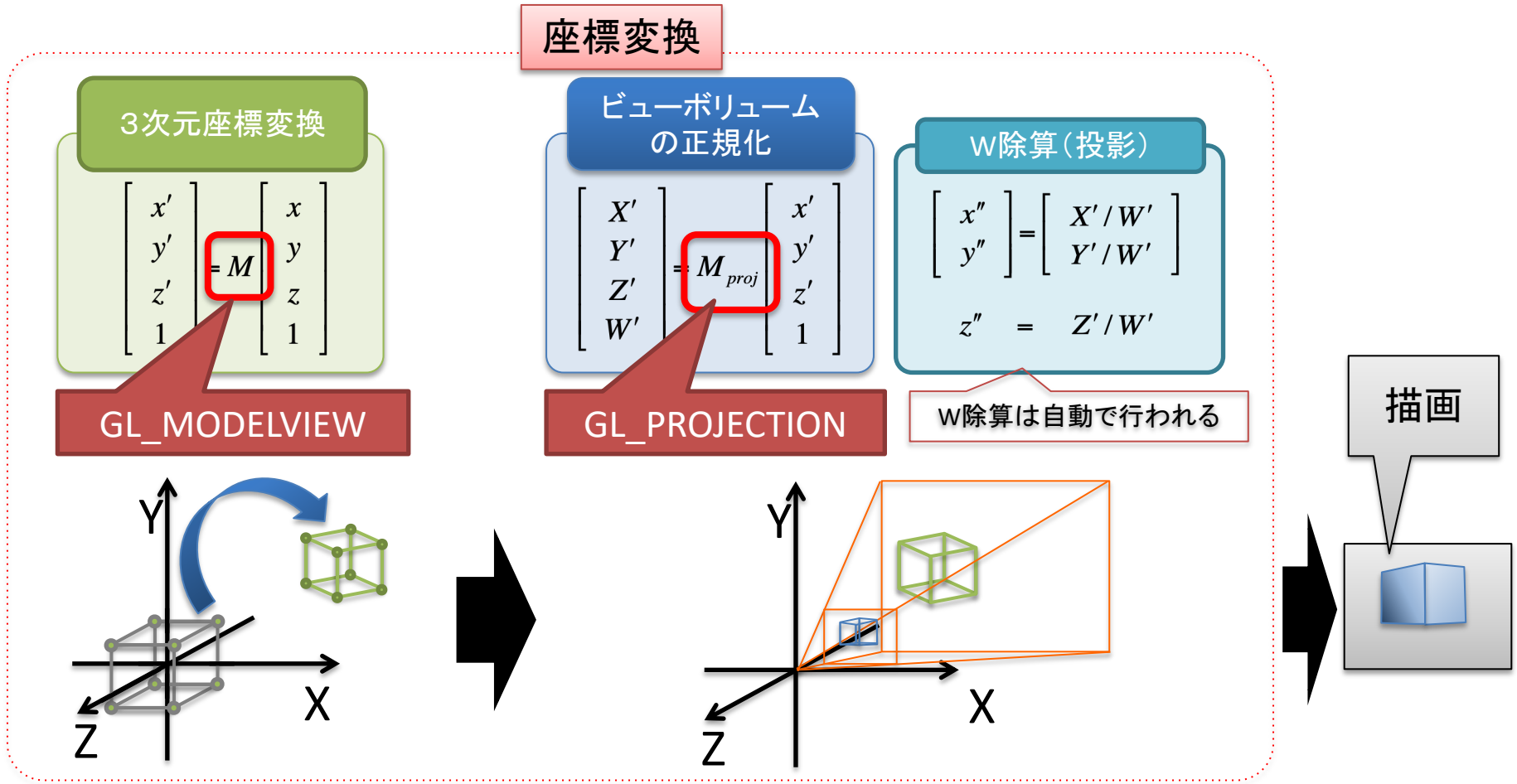
目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

OpenGLの座標変換

- glMatrixMode() でモードを切り替えながら、2つの行列を設定する
  - 3次元座標変換行列（GL_MODELVIEW）
  - 投影行列(GL_PROJECTION)



OpenGLの3次元座標変換

- 座標変換行列を生成する関数が用意されている
  - 単位行列
    - glLoadIdentity()

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - 平行移動
    - glTranslatef(float trans_x, float trans_y, float trans_z)

$$\begin{bmatrix} 1 & 0 & 0 & trans_x \\ 0 & 1 & 0 & trans_y \\ 0 & 0 & 1 & trans_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - 拡大縮小
    - glScalef(float scale_x, float scale_y, float scale_z)

$$\begin{bmatrix} scale_x & 0 & 0 & 0 \\ 0 & scale_y & 0 & 0 \\ 0 & 0 & scale_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - Z軸回転
    - glRotatef(float angle, 0.0f, 0.0f, 1.0f,)

$$\begin{bmatrix} \cos(angle) & -\sin(angle) & 0 & 0 \\ \sin(angle) & \cos(angle) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - X軸回転
    - glRotatef(float angle, 1.0f, 0.0f, 0.0f)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(angle) & -\sin(angle) & 0 \\ 0 & \sin(angle) & \cos(angle) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - Y軸回転
    - glRotatef(float angle, 0.0f, 1.0f, 0.0f)

$$\begin{bmatrix} \cos(angle) & 0 & \sin(angle) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(angle) & 0 & \cos(angle) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

行列の設定例(1)

- 行列設定モードを、座標変換モードにする

```
//座標変換モードにする
glMatrixMode(GL_MODELVIEW);

 // 行列設定
glLoadIdentity();
glTranslatef( 0.0f, 0.0f, -5.0f );
glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );

//描画
glutSolidTeapot( 1.0f );
```

行列の設定例 (2)

- 行列設定を行う（行列が右にかかる）

```
//座標変換モードにする
glMatrixMode(GL_MODELVIEW);

 // 行列設定
glLoadIdentity();
glTranslatef( 0.0f, 0.0f, -5.0f );
glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );

//描画
glutSolidTeapot( 1.0f );
```

初期化

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

移動

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

回転

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 30° & 0 & \sin 30° & 0 \\ 0 & 1 & 0 & 0 \\ -\sin 30° & 0 & \cos 30° & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

行列の設定例 (3)

- 描画する（行列を頂点座標の左にかける）

```
//座標変換モードにする
glMatrixMode(GL_MODELVIEW);

 // 行列設定
glLoadIdentity();
glTranslatef( 0.0f, 0.0f, -5.0f );
glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );

//描画
glutSolidTeapot( 1.0f );
```

初期化

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

移動

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

回転

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 30^\circ & 0 & \sin 30^\circ & 0 \\ 0 & 1 & 0 & 0 \\ -\sin 30^\circ & 0 & \cos 30^\circ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

行列の設定例 (4)

- 変換は、頂点に近い最後の行列から反映される

```
//座標変換モードにする
glMatrixMode(GL_MODELVIEW);

 // 行列設定
glLoadIdentity();
glTranslatef( 0.0f, 0.0f, -5.0f );
glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );

//描画
glutSolidTeapot( 1.0f );
```

初期化

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

移動

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

回転

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 30^\circ & 0 & \sin 30^\circ & 0 \\ 0 & 1 & 0 & 0 \\ -\sin 30^\circ & 0 & \cos 30^\circ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.移動　1.回転

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5.0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 30^\circ & 0 & \sin 30^\circ & 0 \\ 0 & 1 & 0 & 0 \\ -\sin 30^\circ & 0 & \cos 30^\circ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

記述順による描画の違い

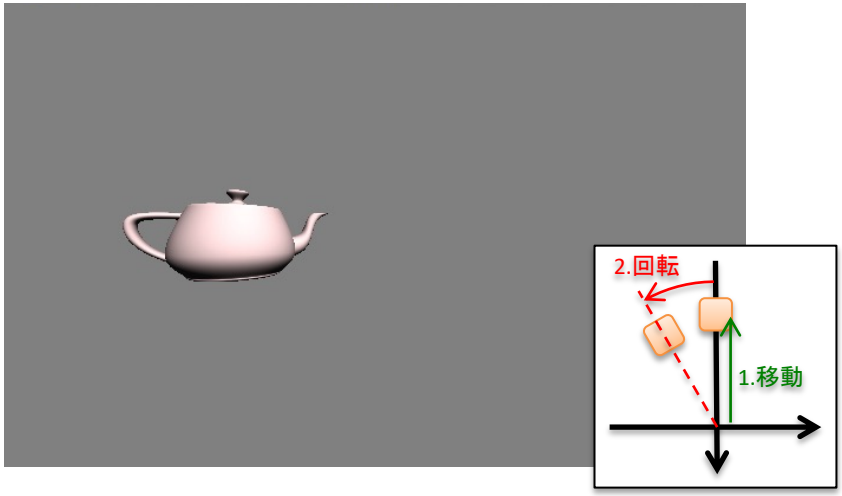



```
//行列を3次元座標変換モードにする
glMatrixMode(GL_MODELVIEW);

 // 行列設定
glLoadIdentity();
glTranslatef( 0.0f, 0.0f, -5.0f );
glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );

//描画
glutSolidTeapot( 1.0f );
```

2.移動
1.回転




```
//行列を3次元座標変換モードにする
glMatrixMode(GL_MODELVIEW);

 // 行列設定
glLoadIdentity();
glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );
glTranslatef( 0.0f, 0.0f, -5.0f );

//描画
glutSolidTeapot( 1.0f );
```

2.回転
1.移動

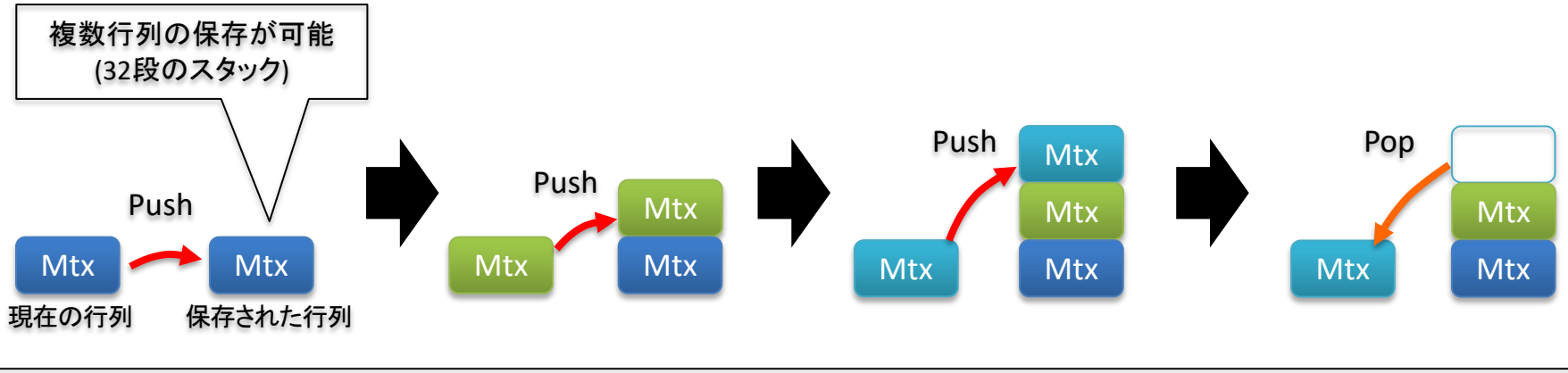
その他の座標変換用関数(参考)

- 行列の置き換え
  - glLoadMatrix(const float *m)
    - 現在の行列の値を、float配列mに置き換える(**並びに注意**)

$$M = \begin{bmatrix} m[0] & m[4] & m[8] & m[12] \\ m[1] & m[5] & m[9] & m[13] \\ m[2] & m[6] & m[10] & m[14] \\ m[3] & m[7] & m[11] & m[15] \end{bmatrix}$$

- 行列の保存と読み出し(下図)
  - glPushMatrix( )
    - 現在の行列を保存する
  - glPopMatrix( )
    - 現在の行列を保存された行列に置き換える

3回Pushした後に、1回Popする場合



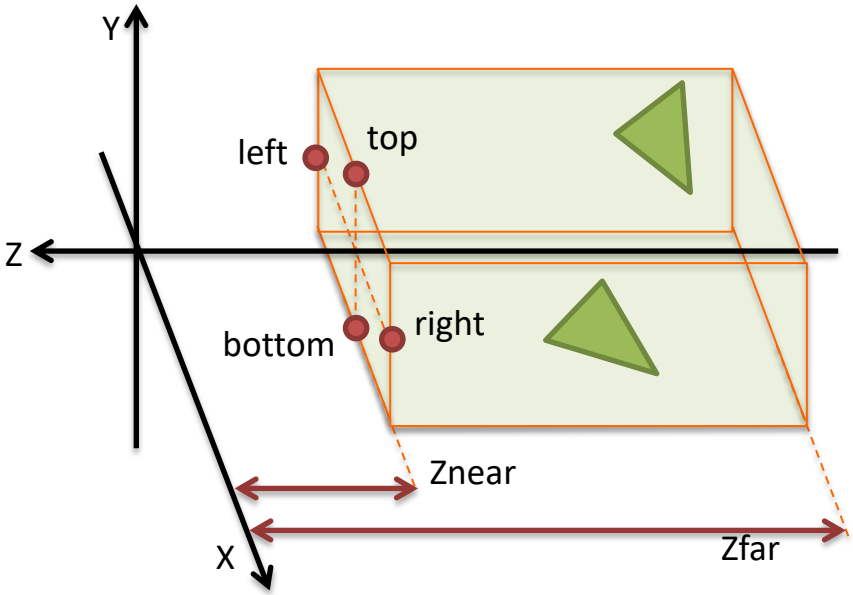
目次

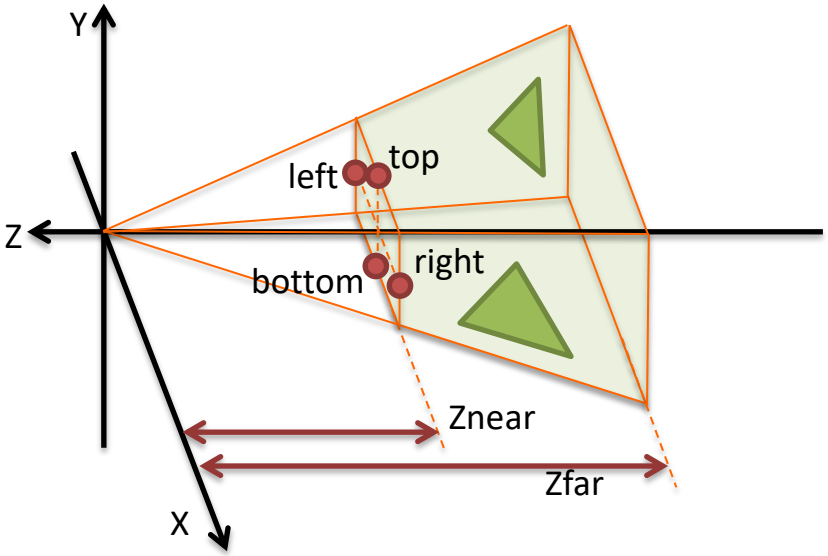
- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

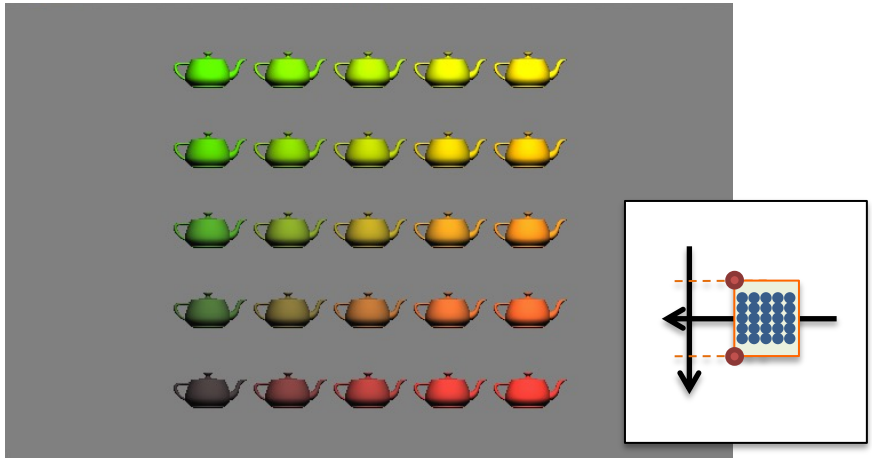
OpenGLの投影変換

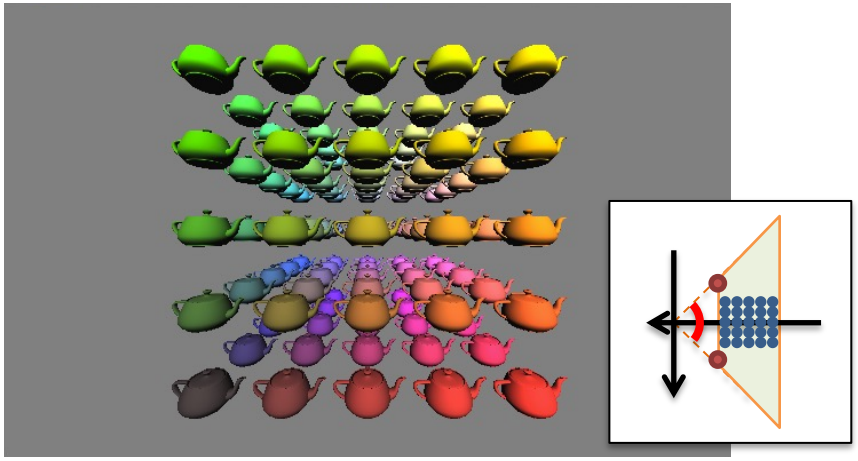
- ビューボリュームを定義する関数が用意されている
  - 平行投影（下図左）
    - glOrtho (double left, double right, double bottom, double top, double Znear, double Zfar)
  - 透視投影（下図右）
    - glFrustum(double left, double right, double bottom, double top, double Znear, double Zfar)





投影方法よる描画結果の違い





```
//行列を投影変換モードにする
glMatrixMode(GL_PROJECTION);

 // 平行投影
glOrtho( -0.75,     // left
         +0.75,     // right
         -0.5,      // bottom
         +0.5,      // top
         1.0,       // z near
         2.5        // z far
        );
```

出力が横長画像なので、ビューボリュームも横長

```
//行列を投影変換モードにする
glMatrixMode(GL_PROJECTION);

 // 透視投影
glFrustum( -0.75,     // left
           +0.75,     // right
           -0.5,      // bottom
           +0.5,      // top
           1.0,       // z near
           2.5        // z far
          );
```

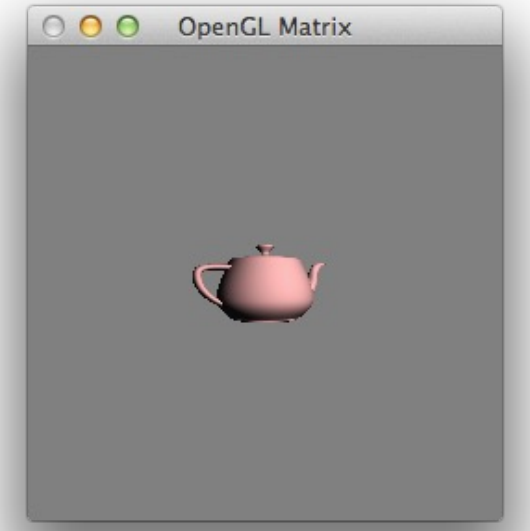
目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

サンプルプログラム

- 所定の場所からzipをコピー&解凍
- ソリューションファイル(*.sln)を開く
- ビルドして実行
  - ウィンドウが表示される（右図）



プログラム解説(1)

```
#include <stdio.h>
#include <GL/glut.h>

static const int WIN_WIDTH  = 256; // window width (pixel)
static const int WIN_HEIGHT = 256; // window height (pixel)

void Initialize();
void Display();

////////////////////////////////////////  main関数  ////////////////////

int main( int argc, char *argv[] ) {

    glutInit( &argc, argv ); // init GLUT
    glutInitWindowSize( WIN_WIDTH, WIN_HEIGHT ); // set window size
    glutInitDisplayMode (GLUT_RGBA | GLUT_DEPTH);  // framebuffer has RGB+A and Depth-buffer
    glutCreateWindow( "OpenGL Matrix" );  // Create OpenGL window

    Initialize(); // init GL environment
    glutDisplayFunc( Display );   // set display loop function

    // start rendering
    glutMainLoop();

    return 0;
}

////////////////////////////////////////////////////////////////////////////////
```

OpenGLの関数定義読み込み

ウィンドウサイズの設定値の宣言

初期化関数、描画関数の宣言

OpenGLそのもの初期化

ウィンドウサイズの設定

バッファはRGB+αと、深さ情報用のバッファとを設定する

ウィンドウ生成

描画設定の初期化

描画関数の登録

描画開始

プログラム解説(2)

初期化関数
（最初に一回呼ばれる）

```
// init
void Initialize() {
   glClearColor( 0.5f, 0.5f, 0.5f, 1.0f ); // set background color: Red,Green,Blue,Alpha

   glEnable(GL_DEPTH_TEST); // enable depth test

   // set projection matrix
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   glFrustum( -1.0f, +1.0f,   // xrange -1~+1
                -1.0f, +1.0f,   // yrange -1~+1
                 1.0f, +100.0f ); // distance range 1~100 (Z=-1.0~-100.0)
   /**
   glOrtho( -1.0f, +1.0f,   // xrange -1~+1
               -1.0f, +1.0f,   // yrange -1~+1
                1.0f, +100.0f ); // distance range 1~100 (Z=-1.0~-100.0)
   **/

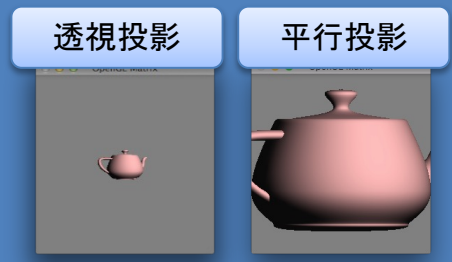
   // set light source information
   glEnable(GL_LIGHTING); // enable lighting

   GLfloat pLightPos[4]   = { 2.5f, 2.0f, 0.0f, 1.0f }; // light's position (x,y,z,1)
   GLfloat pLightColor[4] = { 1.0f, 1.0f, 1.0f, 1.0f }; // light's color (rgba)

   glEnable(GL_LIGHT0); // enable light source
   glLightfv(GL_LIGHT0, GL_POSITION, pLightPos);  // set light's position
   glLightfv(GL_LIGHT0, GL_DIFFUSE, pLightColor); // set light's color
}
```

背景色の設定
RGB+αチャネルを 0.0~1.0 で指定
αチャネルは「不透明度」の値
通常は1.0(完全不透明)を指定

投影行列を設定
透視投影(glFrustum)が設定されている
平行投影(glOrtho)を設定すれば、
描画結果の違いが確認できる



OpenGLの陰影付け機能を有効にする

陰影付けに使う光源の位置と色の定義

光源を有効にして、位置と色を設定する

プログラム解説(3)

描画関数
（描画の度に呼ばれる）

```
// draw
void Display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); // clear framebuffer and depth-buffer

    // set teapot color
    GLfloat pTeapotColor[] = { 1.0f, 0.7f, 0.7f, 1.0f }; // teapot color (rgba)
    glMaterialfv(GL_FRONT, GL_DIFFUSE, pTeapotColor);

    glMatrixMode(GL_MODELVIEW); // set matrix mode to 3d-transformation

    glLoadIdentity();
    glTranslatef( 0.0f, 0.0f, -5.0f );     // 2. translate
    glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );  // 1. rotate
    /**
    glRotatef( 30.0f, 0.0f, 1.0f, 0.0f );  // 2. rotate
    glTranslatef( 0.0f, 0.0f, -5.0f );     // 1. translate
    **/

    glutSolidTeapot( 1.0f ); // draw teapot

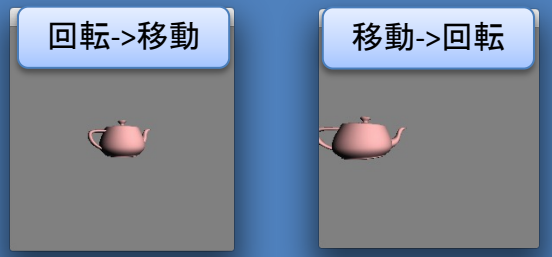
    glFlush(); // end of display routine
}
```

画面初期化（背景色で塗りつぶす）

Teapotの色設定
RGBαを 0.0~1.0 で指定

3次元座標変換行列の設定

回転と並行移動の順番を入れ替えると、描画結果が変わる



Teapotの描画
(引数はモデルの大きさ)

ここまでの記述を描画する

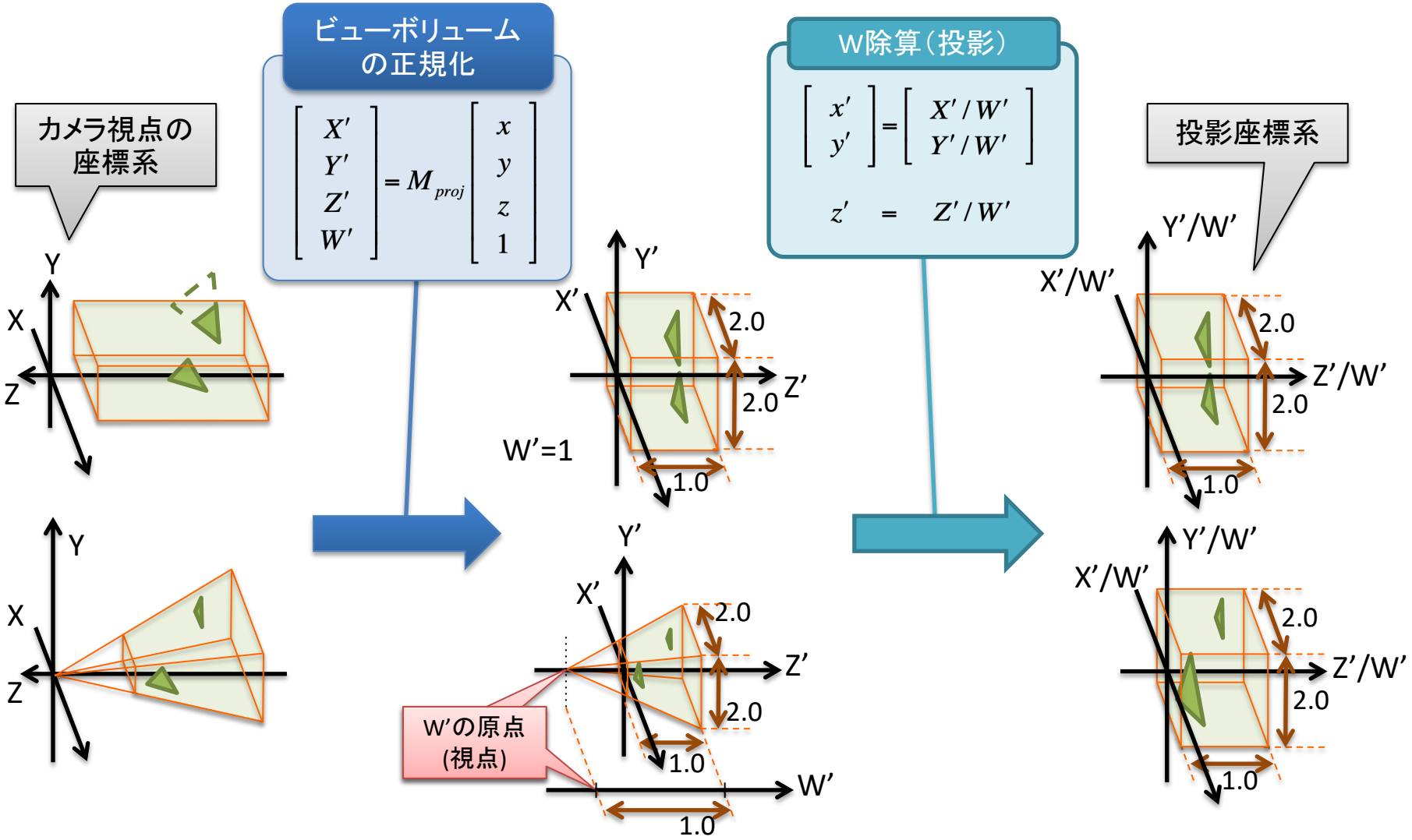
目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

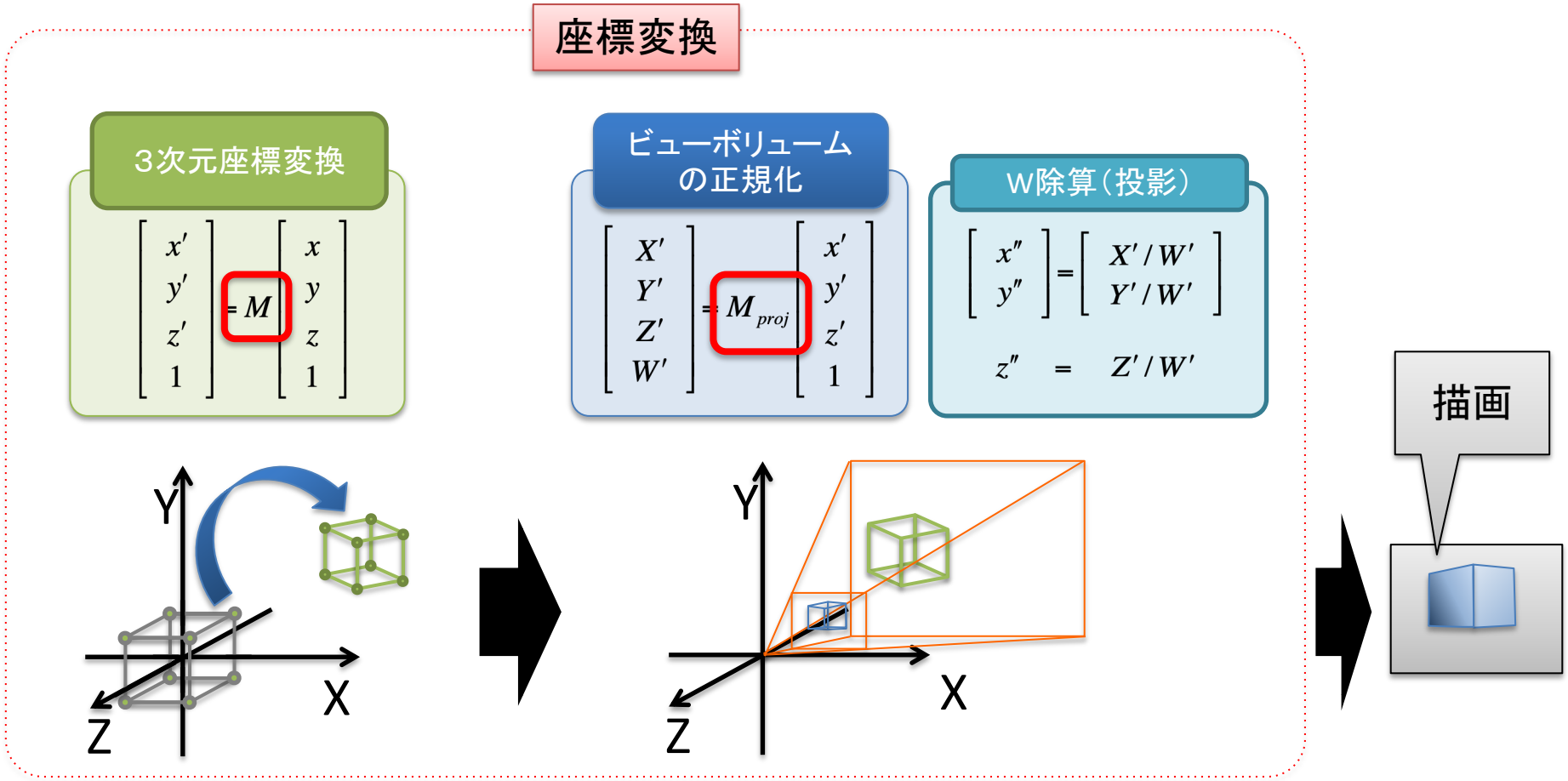
投影変換のまとめ

- 投影変換には、平行投影と透視投影の2種類がある
- 投影変換によって、ビューボリュームを正規化された直方体に変換する
  - 具体的には、投影行列の掛け算と、W除算を行う



座標変換のまとめ

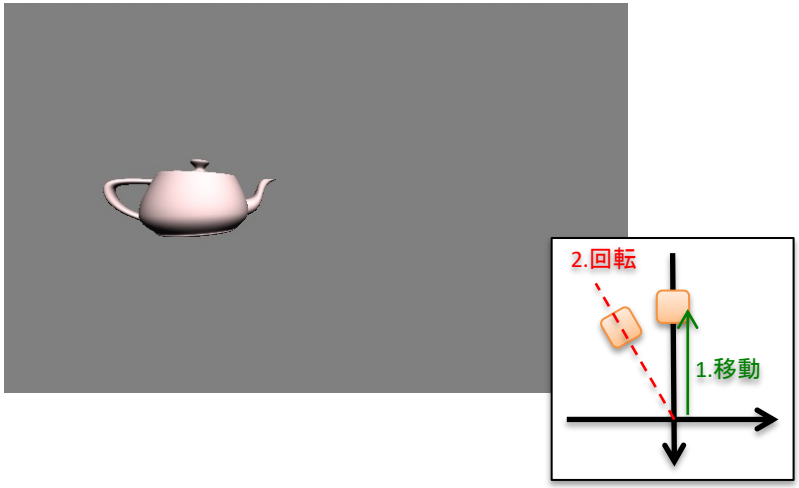
- 3次元座標変換は、行列の積で行う
  - 同次座標によって平行移動、拡大縮小、回転が行列1つに集約できる
- 投影は、行列の積とW除算で行う
  - W除算による投影で、2次元座標が決まる



OpenGLの座標変換のまとめ

- OpenGLでは、モードを切り替えながら行列を設定する
  - 3次元座標変換行列（GL_MODELVIEW）
    - 平行移動、拡大縮小、XYZ軸回転を組み合わせる
    - 順序に注意する（下図）
    - 最後に記述した変換から適用される
  - 投影行列(GL_PROJECTION)
    - 平行投影、透視投影、両方が設定できる





目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

単元課題

- ScombZの「課題」から提出してください
  - ファイル名は、**cg_01_(学籍番号).docx/pdf**
    - 例えば、cg_01_AL12345.docx
    - 学籍番号と名前の情報を、ファイル内容に含めてください
  - ファイル形式
    - Word(docx)、又はpdf

- 締め切り
  - 10月26日 9:00 （次回授業開始まで）

目次

- 投影変換
  - 座標変換の目的と概要
  - 平行投影
  - 透視投影
  - まとめ
- OpenGLの座標変換
  - 3次元座標変換
  - 投影変換
  - サンプルプログラム
- まとめ

- 単元課題
- ミニッツペーパー

ミニッツペーパー

- 「今日の授業で理解できたこと、または分からなかったこと」
  - 授業の参考にするので、白紙での提出は避けてください
  - 例えば、分からなかった方が多かった内容は、そのあとの授業で補足説明をします

- 「授業への要望、質問など」
  - 特になければ、白紙でも構いません

次回はポリゴンレンダリングエンジン



	授業計画	授業時間外課題（予習および復習を含む）	必要学習時間
	トロダクション　＋OpenGL演習（環境構築）	予習：ベクトル，行列，C言語の基礎の確認	60分
		復習：ベクトル，行列の不明点の理解，OpenGLテストプログラムの動作確認	60分
	変換：2次元座標変換、3次元座標変換	予習：教科書 2-1「2次元座標変換」2-2「3次元座標」	60分
		復習：座標変換の行列演算と同次座標の理解	60分
	変換：OpenGL演習（2次元座標変換、3次元座標変換）	予習：c言語の基礎の確認	60分
		復習：演習用プログラムの理解	160分
4.	座標変換：投影変換、座標変換のまとめ	予習：教科書 2-3「投影」	60分
		復習：3次元の座標変換行列，投影行列の理解	60分
5.	ポリゴンレンダリングエンジン：座標系変換，ラスタライズ，シェーディング	予習：教科書 2-4「ビューイングパイプライン」 2-5「描画パイプライン」4-3「シェーディング」	60分
		復習：情報から画像が生成される過程の理解	60分
		課題提出（座標変換）	80分
	ズ，シェーディング）	）	
		復習：演習用プログラムの理解	160分
7.	ポリゴンレンダリングエンジン：テクスチャマッピング，Zバッファ	予習：教科書 4-5「マッピング」	60分
		復習：画像をポリゴンモデルに貼る内部動作の理解	60分
8.	ポリゴンレンダリングエンジン：OpenGL演習（テクスチャマッピング）	予習：OpenGL参考サイト（ http://www.wakayama-u.ac.jp/~tokoi/opengl/libglut.html ）	60分
		復習：演習用プログラムの理解	160分